



ChillZone



**Institut universitaire de technologie de Marne-la-Vallée
Département Informatique**

Rapport Final “ChillZone”

Groupe FC Zen

Helena CHEVALIER
Kellian BREDEAU
Thivakar JEYASEELAN
Elias LAHLOUH
Loïc MAURITIUS
Taha SEFOUDINE

Porteurs du projet

Mme Juliette CLAISSE
Mme Rada MOGLIACCI

Enseignant référent : Mr Olivier CHAMPALLE

2024 - 2025

Sommaire

Sommaire	2
Table des illustrations	3
Table des annexes	3
Glossaire	5
Introduction	9
1. Recueil des besoins	11
1.1 Profils métier	11
1.2 Besoins des différents profils métiers	13
1.3 Fonctionnalités identifiées	15
1.4 Démarche du recueil des besoins	20
2. Spécifications fonctionnelles et techniques	21
2.1 Choix technologiques	21
2.2 Langages, frameworks et outils utilisés	22
2.3. Explication des décisions technologiques	24
2.4. Verrous techniques du développement	26
3. Conception et prototypage	29
3.1 Maquettes de l'application	29
3.2 Architecture du système	48
3.3 Conception de la base de données	51
4. Développement	55
4.0. Méthodologie de projet appliquée	55
4.1. Étapes de développement	57
4.2. Défis rencontrés	59
4.3. Solutions apportées	59
4.4. Code source et démonstration finale	60
5. Tests et validation	62
Conclusion et perspectives	65
Annexes	67
Ressources complémentaires (lien vers les documents annexes)	67

Table des illustrations

Illustration 1 : Matrice de faisabilité des fonctionnalités	15
Ecran d'exemple n°1-4: Pages de connexion	29
Ecran d'exemple n°5-7: Accueil, Notification et FAQ	30
Ecran d'exemple n°8: Barre de Navigation	30
Ecran d'exemple n°9 : Header de la page principale	30
Ecran d'exemple n°10-15: Pages de réservation de salle	32
Ecran d'exemple n°15-18: Pages de compte utilisateur	33
Ecran d'exemple n°19: Calendrier	33
Ecran d'exemple n°20: Page de réservation de repas	34
Ecran d'exemple n°25-27: Pages de réservations de repas	36
Ecran d'exemple n°28-29: Pages de navigation 2D et en réalité augmentée	37
Ecran d'exemple n°30-33 : Pages de connexion des administrateurs sur PC	39
Ecran d'exemple n°34-36: Pages de connexion du restaurateur sur PC	40
Ecran d'exemple n°37-43: Pages des fonctionnalités de l'administrateur sur PC	43
Ecran d'exemple n°44-47: Pages des fonctionnalités du restaurateur sur PC	46
Ecran d'exemple n°48(-49): Page supplémentaire du super-administrateur sur PC	47
Illustration 2 : Architecture des dossiers	49
Illustration 3 : Graphique montrant la répartition globale des tickets par types de tickets (étiquettes)	56
Illustration 3 : Capture d'écran représentant l'architecture des dossiers en Front-end (web et mobile)	60
Illustration 4 : Capture d'écran représentant la structure de l'Atomic Design	61
Illustration 5 : Capture d'écran montrant les pull-requests actuellement ouverte, avec le nom du ticket Jira relié, le nom de la page, la partie du projet concerné, la personne assignée et le nombre de commentaires fait en vérification du code.	62
Illustration 6 : Capture d'écran montrant l'obligation d'avoir un réviseur sur chaque branche sans quoi il est impossible de "merge" la branche sur le main.	62
Illustration 6 : schéma explicatif de la méthode Test-Driven Development (TDD)	64

Table des annexes

1 - Cahier des charges fonctionnel	66
2 - Architecture et choix techniques	66
3 - Documentation de l'architecture technique	66
4 - Documentation base de données	66
5 - Maquette du projet	66
6 - Backlog JIRA contenant les user stories	66
7 - Guide d'installation du projet	66
8 - Documentation des tests utilisateurs	66
9 - Statistiques du groupe	66
10 - Playlist des vidéos du guide d'utilisation	66

Glossaire

ADE (Application de gestion des emploi du temps) : Logiciel utilisé pour planifier et gérer les emplois du temps des cours, des enseignants, et des salles, souvent utilisé par les universités et établissements scolaires pour coordonner leurs activités académiques.

API (interface de programmation d'application) : Interface permettant la communication entre différentes applications via des requêtes HTTP.

Atomic Design: Méthode de conception qui décompose les interfaces en éléments fondamentaux (atomes, molécules, organismes) pour créer des systèmes modulaires et cohérents.

Back-office : Interface d'administration d'une application, généralement accessible aux gestionnaires et administrateurs.

Backend : Partie d'une application qui gère la logique métier, la base de données et la communication avec le frontend.

Base de données relationnelle : Système de stockage d'informations structuré en tables et utilisant le langage SQL pour les requêtes.

Box acoustiques : Espaces insonorisés permettant aux étudiants de travailler dans un environnement calme.

BUT (Bachelor Universitaire de Technologie) : Diplôme universitaire en trois ans, proposé en IUT.

ChatBot : IA conversationnelle capable d'interagir avec les utilisateurs via une interface conversationnelle.

CI/CD (Continuous Integration / Continuous Deployment) : Méthodologie permettant d'automatiser l'intégration et le déploiement des applications.

Click and Collect : Service permettant de commander un produit en ligne et de le récupérer sur place.

CIPEN (Centre d'Innovation pour la Performance et l'Environnement Numérique) : Structure dédiée à l'innovation numérique et technologique.

Comitologie : Fréquence et types des réunions durant le projet

Diagramme de Gantt : Représentation graphique permettant de visualiser l'avancement d'un projet et sa planification.

Epic : Grande fonctionnalité d'un projet Agile, découpée en plusieurs tickets ou user stories.

ENT (Environnement Numérique de Travail) : Plateforme centralisant les services numériques d'un établissement scolaire ou universitaire.

Endpoints : Points d'accès spécifiques dans une API REST où des ressources ou des services sont exposés, identifiés par une URL et accessibles via des méthodes HTTP telles que GET, POST, PUT ou DELETE.

FAQ (Foire aux Questions) : Section regroupant les réponses aux questions-réponses fréquentes des utilisateurs.

Frameworks : Structures logicielles facilitant le développement en fournissant des composants et des outils préconçus.

Frigos connectés : Réfrigérateurs intelligents permettant la commande et la récupération de repas de manière automatisée.

Front-End : Partie visible d'une application avec laquelle l'utilisateur interagit directement.

HTTP (HyperText Transfer Protocol) : Protocole de communication permettant le transfert de données sur le web.

IOT (Internet of Things) : Réseau d'objets connectés capables de communiquer entre eux via Internet.

IOS : Système d'exploitation mobile développé par Apple.

IUT (Institut Universitaire de Technologie) : Établissement d'enseignement supérieur proposant des formations professionnalisantes.

JSON (JavaScript Object Notation) : Format de données léger permettant d'échanger des informations entre un client et un serveur.

Lien ICAL : Format permettant de partager et synchroniser des calendriers en ligne.

MCD (Modèle Conceptuel de Données) : Modélisation des données et de leurs relations dans une base de données.

MEA (Modèle Entité-Association) : Schéma représentant les entités d'un système et leurs relations.

Méthode Agile : Approche de gestion de projet favorisant l'adaptabilité, les itérations courtes et la collaboration.

MLD (Modèle Logique de Données) : Représentation détaillée des données avant leur implémentation dans un système de gestion de base de données.

MOA (Maîtrise d'Ouvrage) : Responsable de la définition des besoins et du suivi du projet, généralement côté métier.

MOE (Maîtrise d'Œuvre) : Responsable de la mise en œuvre technique du projet, généralement côté développement.

MySQL : Système de gestion de bases de données relationnelles open-source.

ORM (Object-Relational Mapping) : Outil permettant de manipuler une base de données relationnelle à travers un langage orienté objet.

Plan interactif : Carte numérique dynamique facilitant la navigation dans un établissement.

POC (Proof of Concept) : Prototype fonctionnel permettant de valider une idée ou une technologie avant un développement complet.

Pull Request : Proposition de modification de code soumise pour validation avant son intégration dans un projet Git.

QR Codes : Codes-barres bidimensionnels scannables pour accéder rapidement à des informations ou déclencher des actions.

RA (Réalité Augmentée) : Technologie superposant des éléments virtuels à la réalité à travers un écran ou des lunettes spécifiques.

Rétrospectives de sprint : Réunions en fin de sprint Agile visant à analyser les points forts et les axes d'amélioration.

RH (Ressources Humaines) : Service chargé de la gestion administrative et organisationnelle du personnel.

Sprint : Période de travail définie dans un projet Agile, durant laquelle une série de tâches doit être réalisée.

Synchronisation : Processus d'alignement des données entre plusieurs systèmes ou appareils.

TDD (Test-Driven Development) : Méthode de développement basée sur l'écriture des tests avant la programmation du code.

Tickets : Tâches ou fonctionnalités suivies dans un projet Agile, généralement sur un outil comme Jira.

TIPI (Technologies de l'Information et Patrimoine Immobilier) : Domaine d'application des nouvelles technologies à la gestion du patrimoine immobilier.

UI/UX (User Interface / User Experience) : Ensemble des principes de conception visant à améliorer l'apparence et l'ergonomie d'une interface utilisateur.

UML (Unified Modeling Language) : Langage de modélisation permettant de représenter graphiquement l'architecture d'un système.

VVT (Validation, Vérification & Test) : Ensemble des processus garantissant la qualité et la conformité d'un logiciel.

Introduction

Dans le cadre de notre troisième année de BUT informatique à l'Université Gustave Eiffel, nous menons un projet sur les deux semestres visant à mobiliser nos connaissances et compétences professionnelles.

Ce projet, intitulé "La Chill Zone", s'inscrit au sein du projet TIPI (Technologies de l'Information et Patrimoine Immobilier). Il a pour objectif de répondre à la problématique ci-dessous :

"Comment optimiser les espaces et améliorer l'expérience utilisateur au quotidien à l'IUT ?"

Ce projet est mené en étroite collaboration avec le CIPEN, ce dernier assure les rôles de client et de maîtrise d'ouvrage (MOA) en nous aidant à affiner les besoins et les attentes.

Structuré autour d'un cahier des charges fourni par l'ancien groupe de Masters de la branche Management de l'Innovation, le projet "La Chill Zone" cherche à optimiser les espaces universitaires en modernisant des processus longs et non automatisés. Cela a pour but d'améliorer l'expérience des étudiants au sein de l'IUT de Champs-sur-Marne. Cette modernisation passe donc par une application mobile nommée "Mon IUT".

À terme, ce projet a vocation à s'étendre à l'ensemble de l'université Gustave-Eiffel.

Notre mission principale consiste à concevoir et développer un POC (Proof of Concept) sous la forme d'une application mobile. Ce POC n'est pas une solution finale prête à être déployée, mais une démonstration technique et fonctionnelle permettant de mettre en avant la viabilité et l'intérêt du projet auprès des parties prenantes, notamment les responsables institutionnels. Cette approche vise à justifier le lancement du projet et l'attribution des ressources nécessaires pour le TIPI (immobilier et numérique).

Le développement de ce POC est contraint à un délai de 6 mois (de octobre 2024 à mars 2025) et est réalisé par une équipe en plein apprentissage des compétences nécessaires avec l'ambition de satisfaire des attentes diverses tout en offrant une solution à la fois simple, ludique et intuitive.

Ce projet représente un double défi. Sur le plan technique, il implique des choix technologiques stratégiques, la gestion collaborative via GitHub, une montée en compétence sur des frameworks modernes, et la complexité du développement multiplateforme. Sur le plan organisationnel, il requiert une gestion rigoureuse en méthode agile, avec des interactions régulières avec le professeur et les clients, ainsi qu'une coordination efficace des réunions et des livrables.

Notre équipe est composée de six étudiants, se répartissant le développement de l'application front et back end. Hélène CHEVALIER, assume le rôle de cheffe de projet, Kellian BREDEAU de responsable Back-End, Elias LAHLOUH de responsable Front-End, Loïc MAURITIUS de responsable documentaire, Taha SEFOUDINE de responsable des tests et Thivakar JEYASEELAN de responsable design UI/UX.

L'ensemble de l'équipe est encadré par M. Champalle, enseignant-chercheur en informatique.

Ce rapport final a pour objectif de proposer une version du POC dynamique pour justifier l'intérêt du projet par rapport à son coût RH et financier, ainsi que les avancées faites sur le développement de "Mon IUT".

Il présente les différentes étapes du projet et notre avancement : définition des besoins, conception des premières maquettes, choix des technologies, et mise en œuvre des fonctionnalités initiales. Il s'articule autour de quatre parties principales : analyse et préparation du projet, développement du POC, tests réalisés, et perspectives pour le lancement du projet pour le métier.

1. Recueil des besoins

Pour répondre efficacement à la problématique définie, nous avons à notre disposition un cahier des charges fourni par les élèves de Masters, qui ne reflétait pas suffisamment l'ambition du projet. Il était donc nécessaire de réaliser un nouveau recueil de besoins, phase importante afin de préciser la charge de travail à réaliser en tant que développeur.

Nous avons alors réservé tout le premier mois du projet pour pouvoir effectuer toutes les réflexions avec la MOA et notre professeur via plusieurs réunions pour bien définir les besoins.

Il y a d'ailleurs eu un changement de MOA au début du projet ce qui a retardé un peu la procédure de recueil des besoins, mais nous a permis d'avoir des retours davantage orientés sur l'avenir du projet plus que sur la forme.

En explorant les différentes fonctionnalités décrites dans le cahier des charges originel, et en utilisant la maquette fournie par la MOA, nous avons analysé et approfondi chaque point afin de les rendre plus clairs et concis. Cette démarche a permis de revoir et d'enrichir le cahier des charges pour en faire un cahier des charges fonctionnel (axé uniquement sur les fonctionnalités à réaliser), validé par la MOA.

Étant donné que notre projet repose sur la création d'un POC, nous avons identifié des alternatives pour optimiser les résultats avec les moyens et connaissances à notre disposition. Par exemple, avec la MOA nous avons décidé de remplacer le ChatBot par une "Foire aux questions" et de simplifier la fonctionnalité de réalité augmentée pour l'adapter à nos contraintes de temps et de compétences.

Le recueil des besoins a été réalisé principalement lors de réunions, en particulier avec l'aide de Juliette CLAISSE et Rada MOGLIACCI, qui ont su répondre à nos questions pour affiner les besoins. De plus, à la demande de la MOA, qui souhaitait connaître les attentes et le ressenti des utilisateurs vis-à-vis des fonctionnalités envisagées, nous avons également élaboré un questionnaire, envoyé à notre classe ainsi qu'à d'autres membres de l'IUT Gustave Eiffel.

Le lien vers le cahier des charges fonctionnel réalisé par notre équipe est disponible en annexe 1 - Cahier des charges.

1.1 Profils métier

Pour concevoir le POC, il nous a fallu déterminer les différents profils d'utilisateurs de l'application cible. Le cahier des charges classique nous a tout d'abord présenté le profil de l'utilisateur principal ; il s'agit des étudiants. Une réflexion a été menée pour rendre l'application accessible aux professeurs et au personnel de l'université. Cependant, leurs cas d'utilisation étant similaires à ceux des étudiants, ils ont été intégrés à la catégorie générale des utilisateurs.

Ensuite, en examinant en détail le cahier des charges fourni avec les fonctionnalités attendues, nous nous sommes interrogés sur la gestion de ce type

d'applications au sein de l'université. Il existe différents types d'administrateurs au sein de l'université pour gérer le back office de l'application ainsi que certains services spécifiques en lien avec les besoins utilisateurs.

Enfin, concernant une fonctionnalité spécifique demandée, à savoir la réservation de repas en Click and Collect, nous avons longuement réfléchi à son fonctionnement. Cette fonctionnalité repose sur l'utilisation de frigos connectés approvisionnés chaque matin et gérés par un restaurateur du CROUS, ce qui a nécessité l'ajout d'un profil métier dédié. De plus, après discussion avec la MOA, il est apparu que le Click and Collect pourrait également être étendu à des restaurants situés à proximité.

Cette réflexion sur les profils métier nous a permis de nous pencher sur des cas concrets d'utilisation de l'application et d'affiner les besoins spécifiques liés à chaque fonctionnalité.

Au terme de cette réflexion, nous avons pu identifier les profils métiers suivants :

- **L'utilisateur** : L'utilisateur est divisé en deux sous-types : Étudiant et Professeur. La plateforme lui permet de consulter des informations, accéder à des ressources de son établissement, et effectuer des réservations de repas ou de salles. Il représente à la fois les étudiants mais aussi les professeurs et tout autre personnel de l'université.
- **L'administrateur** : L'administrateur est responsable de la gestion globale du système de réservation des salles de son établissement, de la gestion des données de son établissement comme les plans. Chaque établissement aura plusieurs administrateurs, car la gestion des comptes utilisateurs de son établissement ou la gestion des salles disponibles ne dépend pas que d'une seule personne dans l'organisation de l'université.
- **Le restaurateur** : Le restaurateur est responsable du service de restauration qu'il propose aux utilisateurs de l'IUT. Il doit pouvoir alimenter les frigos connectés en renseignant le nombre de repas ou de produits autres, et gérer les produits des frigos connectés. Par la suite, si cette fonctionnalité s'étend aux points de restaurations proches, il pourra analyser et gérer les commandes passées par l'application web.
- **Le super administrateur** : Le super-administrateur est responsable de la gestion globale du système de tous les établissements. Il est important de différencier ce rôle de celui du super-administrateur Django, qui est un rôle spécifique à la gestion technique de l'application dans l'interface d'administration de Django. Ici ce super-administrateur joue un rôle de supervision sur les comptes liés à son établissement.

Nous nous sommes interrogés sur le profil de rédacteur pour la gestion des actualités, mais ce profil a été supprimé à la demande de la MOA.

1.2 Besoins des différents profils métiers

Les profils métiers identifiés, nous nous sommes penchés sur les besoins concrets de chaque type d'utilisateur, nous avons détaillé les différents cas d'usage de l'application pour anticiper les conflits ou problèmes rencontrés lors de la phase de qualification.

Nous avons tenu des réunions au sein de l'équipe, ainsi qu'avec le professeur référent et la MOA, pour affiner au maximum chaque besoin pour chacune des fonctionnalités demandées pour ce projet. Les fonctionnalités à mettre en avant ont d'abord été celles concernant le bien-être des étudiants, à savoir la réservation de box acoustiques pour pouvoir travailler plus sereinement, ou encore la réservation de repas en click and collect afin de gagner du temps sur le temps du repas.

En ce qui concerne les utilisateurs, la MOA a insisté fortement pour développer un POC qui respecte les normes d'accessibilité. Le POC doit être conçu de manière à être accessible à tous les types d'étudiants, y compris ceux en situation de handicap.

L'application mobile sert à regrouper certains besoins de l'étudiant, notamment l'accès rapide à son emploi du temps ou encore la consultation de ses cours et des actualités de son établissement.

Elle répond également à de nouveaux besoins tels que rendre la restauration plus rapide pour respecter les temps de repas, s'orienter dans l'université de manière efficace ou encore réserver des box acoustiques pour travailler.

Les notifications du mobile sont idéales pour implémenter des fonctionnalités telles que la réservation de repas, de box acoustiques, ou la consultation du calendrier. Elles permettent une gestion rapide et pratique, surtout en situation de mobilité (peu pratique avec un ordinateur).

Pour les besoins du back-office, nous avons opté pour une application web accessible via Internet, car la gestion de tableurs sur téléphone est difficile et peu pratique. Le restaurateur, le super-administrateur et l'administrateur d'établissement sont donc sur une interface web. En choisissant le web, on garantit une meilleure ergonomie. La lisibilité est améliorée, et l'efficacité dans la gestion des tâches administratives est accrue. Cela a été un choix avec la MOA.

Voici les différentes grandes fonctionnalités avec besoins identifiés :

Authentification et Sécurité

Les utilisateurs ont besoin d'une connexion sécurisée pour accéder à l'application et protéger leurs données personnelles. Cela inclut la possibilité de récupérer un mot de passe oublié, de mettre à jour leurs informations personnelles, et d'être alertés en cas d'activités suspectes ou de changements critiques. Nous allons également mettre en place des flux sécurisés pour la connexion à la base de données.

Gestion des réservations de repas et paiement

Les utilisateurs souhaitent aussi vérifier la disponibilité des repas, effectuer des commandes en toute sécurité, et suivre leur statut. Cette section répond aussi aux besoins de simplifier le retrait avec des QR codes.

Réservation des salles

Ils ont besoin d'un système clair pour réserver des salles, valider leur présence, et signaler d'éventuels problèmes. Ils souhaitent également intégrer automatiquement leurs réservations à leur agenda personnel.

Notifications

Les utilisateurs, restaurateurs, et administrateurs attendent d'être informés en temps réel des changements importants comme les confirmations, les annulations, ou les rappels. Ces notifications améliorent la réactivité et l'expérience utilisateur.

Gestion de l'emploi du temps et de l'agenda

Les utilisateurs cherchent une solution centralisée pour consulter et organiser leurs activités. Ils souhaitent synchroniser leur emploi du temps avec ADE et disposer d'outils pour visualiser et filtrer les événements facilement.

Plan interactif et recherche sur le plan

Les utilisateurs souhaitent naviguer intuitivement sur un plan interactif pour trouver des bâtiments, des points d'intérêt, et des box acoustiques. La possibilité de scanner des QR codes pour des informations précises et des indications de localisation est également essentielle.

Vie étudiante (Réseaux sociaux et foire aux questions)

Les utilisateurs ont besoin d'être informés sur les actualités de leur établissement et notifiés de celles-ci. Ils ont également besoin d'assistance pour naviguer dans l'application et pouvoir poser leurs propres questions dessus.

Gestion de l'administrateur d'établissement

Les administrateurs doivent pouvoir gérer efficacement les comptes, les réservations, et les espaces de l'établissement. Cela inclut l'accès à des statistiques détaillées, l'ajout de nouvelles ressources, et la gestion des signalements pour garantir une organisation optimale.

Gestion du restaurateur

Enfin, nous avons pensé à rajouter des besoins restaurateurs pour montrer la possibilité d'évolution avec des futurs restaurants partenaires et plus généralement, pour la gestion des frigos connectés. Les restaurateurs doivent

suivre et gérer leurs stocks, rendre un point de restauration indisponible temporairement, et surveiller les commandes en cours. Ils ont également besoin d'outils pour analyser les données et améliorer leur service.

1.3 Fonctionnalités identifiées

En analysant les profils métiers, en interagissant avec les clients et en respectant les exigences définies dans le cahier des charges fonctionnel, nous avons pu ajuster les fonctionnalités en fonction des besoins et en identifier de nouvelles.

Dans le cadre du POC, nous savions que nous étions limités par rapport à la demande du client. Nous avons donc évalué la faisabilité de chaque fonctionnalité dans le temps imposé par le projet. Cette évaluation s'est faite via une matrice de faisabilité en tenant compte de l'impact de la fonctionnalité à savoir son importance au sein du projet défini avec la MOA et les efforts fournis pour y arriver.

Voici ci-dessous la liste des fonctionnalités classées par score d'impact :

	Résumé	Score d'impact	Impact	Effort	Catégorie	Priorité
1	Réservation de box acoustiques	100	4	3	Réservations	Haute
2	Rappels de réservation	88	5	3	Réservations	Moyenne
5	Consultation des actualités de l'IUT	88	3	3	Informations étudiantes	Faible
3	Réservation de repas en Click and Collect	86	5	4	Réservations	Haute
6	Consultation du planning de cours/emploi du temps	83	5	5	Informations étudiantes	Haute
7	Notifications des changements d'emploi du temps	75	4	4	Informations étudiantes	Moyenne
8	Accès aux mails et ENT	66	3	4	Informations étudiantes	Moyenne
4	Interface de paiement sécurisé pour les repas	60	4	5	Réservations	Faible
9	Gestion de l'agenda personnel	60	4	5	Informations étudiantes	Haute
10	Plan interactif avec réalité augmentée	46	4	5	Accessibilité	Haute
11	Fonctionnalité de recherche dans le plan interactif	40	3	5	Accessibilité	Faible

Illustration 1 : Matrice de faisabilité des fonctionnalités

Le tableau nous a permis de trier les fonctionnalités par ordre d'importance. Nous les avons ensuite réparties sous forme d'epic afin de préparer le développement en méthode Agile. La méthode Agile est une pratique de développement qui consiste à développer petit à petit chaque fonctionnalité et

de faire des retours constants sur l'avancée du projet. Notre groupe a adopté cette méthodologie très répandue dans les entreprises modernes, car elle permet une mise en place rapide et efficace, ainsi qu'une gestion efficace du développement par fonctionnalités (nous avons basé notre cahier des charges fonctionnels sur ces principes).

Les accès aux mails et à l'ENT ou encore la gestion de l'agenda personnel n'ont pas été retenus étant donné qu'ils n'apportent pas de plus value au projet. Nous avons donc détaillé chaque fonctionnalité et créé différents Epic qui regroupent un ensemble de sous-fonctionnalités spécifiques à chaque action effectuée par les profils métiers.

Epic : Authentification et Sécurité

Le regroupement de cet Epic vise à centraliser les fonctions essentielles pour garantir un accès sécurisé et une protection des données personnelles des utilisateurs. Il englobe des fonctionnalités comme la connexion sécurisée par email et mot de passe, la réinitialisation du mot de passe, la gestion des informations personnelles, ainsi que des alertes en cas d'activités suspectes. Cela permet aux utilisateurs d'accéder à l'application en toute sécurité tout en ayant la certitude que leurs données sensibles sont protégées et conformes aux normes de sécurité. Les normes de sécurité mises en vigueur par notre application seront conformes aux exigences du RGPD.

Les utilisateurs, restaurateurs, et administrateurs s'attendent à pouvoir se connecter facilement de manière sécurisée, récupérer leur mot de passe en cas d'oubli, et mettre à jour leurs informations personnelles de manière fluide.

De plus, un niveau de sécurité réseau renforcé sera mis en place pour protéger les échanges de données entre les utilisateurs et l'application, incluant des protocoles de chiffrement pour garantir la confidentialité et l'intégrité des informations transmises.

Composants concernés du système :

- Base de donnée
- Pages de connexion et changement de mots de passe
- Page de compte
- Page de notification

Epic : Gestion de l'administrateur d'établissement

Cet Epic regroupe les fonctionnalités dédiées à la gestion administrative d'un établissement. Il permet aux administrateurs de superviser les comptes utilisateurs, les réservations des espaces reservables, ainsi que de suivre les statistiques et les signalements d'incidents. Cela permet une gestion fluide et centralisée de l'établissement, tout en offrant des outils pour prendre des décisions et appliquer des actions correctives rapidement.

Les administrateurs s'attendent à un tableau de bord centralisé pour gérer les comptes utilisateurs, superviser les réservations, et analyser les statistiques. Ils veulent également pouvoir gérer les signalements et appliquer des mesures disciplinaires comme le blocage de réservations en cas de non-respect des règles.

Composants concernés du système :

- Tableau de bord de l'administrateur
- Tableurs de gestion de l'université

Epic : Gestion du restaurateur

Cet Epic regroupe les outils nécessaires pour que les restaurateurs puissent gérer efficacement les stocks, suivre les commandes en temps réel, et gérer la disponibilité de leurs restaurants ou frigos. Il permet également de proposer des menus spécifiques et d'exporter des données pour les analyser, ce qui facilite la gestion des opérations quotidiennes.

Les restaurateurs attendent une interface simple et efficace pour gérer les stocks, suivre l'état des commandes et ajuster la disponibilité des points de restauration en temps réel. Ils souhaitent également pouvoir exporter des données pour faciliter le reporting et l'analyse de leurs performances.

Composants concernés du système :

- Tableau de bord du restaurateur
- Tableaux de gestion du restaurants ou frigos

Epic : Gestion des réservations de repas et paiement

Cet Epic permet aux utilisateurs de réserver des repas et d'effectuer des paiements en ligne de manière sécurisée. Il inclut la vérification de la disponibilité des repas, la mise à jour des stocks en temps réel pour éviter les erreurs de commande, le récapitulatif avant paiement, ainsi qu'un système de suivi des commandes pour les restaurateurs.

Les utilisateurs s'attendent à pouvoir consulter la disponibilité des repas, passer commande en toute sécurité, et recevoir un récapitulatif avant paiement. Les restaurateurs souhaitent un système de suivi des commandes clair, la génération de QR codes pour faciliter le retrait des repas, et la possibilité de proposer des menus spécifiques (ex : végétarien, halal, sans lactose).

Composants concernés du système :

- Pages de réservations des repas
- Page de paiement
- Page de suivi des commandes (lié au tableau de bord du restaurateur)

Epic : Réservation des salles

Cet Epic simplifie la gestion des réservations de salles en centralisant toutes les étapes, de la consultation des créneaux disponibles à la confirmation de la réservation. Les utilisateurs peuvent aussi signaler des problèmes dans les salles et synchroniser leurs réservations avec leur agenda personnel, ce qui facilite leur gestion du temps. Ce processus est conçu pour assurer une gestion fluide et un meilleur suivi des espaces reservables.

Les utilisateurs s'attendent à pouvoir consulter facilement les créneaux disponibles, réserver une salle, et confirmer leur présence. Ils veulent aussi pouvoir signaler toute irrégularité et synchroniser leur réservation avec leur agenda. Les administrateurs, quant à eux, attendent un outil de suivi efficace des réservations.

Composants concernés du système :

- Pages de réservations des salles
- Page de suivi des réservations

Epic : Notifications

Cet Epic centralise les alertes et notifications nécessaires pour maintenir une bonne organisation et informer les utilisateurs des changements importants. Les notifications incluent des rappels pour les réservations, des alertes en cas de modification ou annulation, et des informations sur la disponibilité des repas. Elles permettent aussi aux administrateurs et restaurateurs de communiquer des annonces importantes à tous les utilisateurs.

Les utilisateurs attendent des notifications claires et en temps réel concernant leurs réservations, rappels et modifications.

Composants concernés du système :

- Page de notifications des utilisateurs
- Page de notifications administrateur et restaurateur

Epic : Gestion de l'emploi du temps et de l'agenda

Cet Epic permet aux utilisateurs de consulter un agenda synchronisé avec leurs cours, activités et réservations, facilitant ainsi la gestion de leur emploi du temps. Il inclut des options de synchronisation avec d'autres outils comme l'ADE pour garantir une organisation optimale et une visibilité complète de leurs événements.

Les utilisateurs attendent une interface simple et synchronisée avec leur emploi du temps, avec des options de filtrage par semaine ou mois et des détails accessibles sur chaque événement.

Composants concernés du système :

- Pages de consultation de l'agenda
- Page d'ajout de lien entre ADE et le calendrier

Epic : Plan interactif et recherche sur le plan

Cet Epic permet aux utilisateurs de naviguer facilement sur un plan interactif en 2D, de localiser des points d'intérêt et de visualiser des informations détaillées via des QR codes. Il facilite la navigation dans les établissements en offrant une vue dynamique de l'agencement des bâtiments et des services disponibles.

Les utilisateurs souhaitent une carte interactive de navigation claire et facile pour se repérer et accéder aux informations sur les services et espaces. La possibilité de scanner des QR codes pour obtenir des détails supplémentaires est également attendue.

Composants concernés du système :

- Pages du plan interactif 2D de son établissement
- Pages du guidage en réalité augmentée

1.4 Démarche du recueil des besoins

Pour réaliser ce recueil de besoins, nous avons rencontré quelques difficultés. Les étudiants en master qui avaient travaillé sur le projet l'année précédente n'ont pas poursuivi cette année en tant que MOA, laissant des documents incomplets et imprécis. Il a donc fallu repartir de zéro avec de nouveaux responsables de projet pour refaire le recueil des besoins et le cahier des charges fonctionnel.

Il était nécessaire de mettre en place plusieurs réunions avec la nouvelle MOA, qui elle aussi prenait connaissance du projet en septembre, ce qui a rallongé notre phase de "prise en main du projet".

Les discussions ont permis de mieux définir les attentes, mais elles ont également mis en lumière certaines zones d'ombre dans les documents existants. Il a donc fallu reformuler et préciser certains besoins, ce qui a retardé le lancement effectif de la phase de conception et de la phase de développement.

Les fonctionnalités ont été également revues selon les contraintes de temps et de connaissances de l'équipe de développement. Par exemple, le ChatBot de l'application qui n'était pas mentionné dans le rapport (mais visible sur les premières maquettes) a été transformé par la MOA en une "foire aux questions", servant le même besoin tout en étant plus réalisable techniquement.

Toutes les réunions et leurs décisions ont été consignées par écrit et sont disponibles dans l'outil Confluence.

Ces échanges ont été essentiels pour garantir que les fonctionnalités à mettre en place correspondent bien aux attentes des utilisateurs finaux et aux objectifs du POC.

De manière globale, ce travail de recueil des besoins nous a donc permis d'enrichir le panel de fonctionnalités en fonction des différents profils métiers identifiés. Tous ces besoins ont été approuvés par la MOA.

2. *Spécifications fonctionnelles et techniques*

Une fois les besoins clairement définis, nous devons donc détailler les besoins utilisateurs en solutions concrètes pour ce POC. Pour ce faire, nous avons traduit ces exigences en fonctionnalités spécifiques, en tenant compte des différents profils utilisateurs et de leurs attentes. Cette démarche a permis de créer une feuille de route technique précise, tout en assurant la flexibilité nécessaire pour intégrer des évolutions futures en fonction des retours des utilisateurs et des contraintes du projet.

2.1 Choix technologiques

Ce qui ressort du recueil des besoins, c'est que bien que l'application mobile est l'outil principal pour l'utilisateur (comme décrit dans le cahier des charges original). Cependant il est plus adapté pour l'administrateur et le restaurateur en back-office d'avoir une interface web.

Il nous a donc fallu concevoir à la fois une solution web et une solution mobile, en utilisant deux technologies récentes, proches mais différentes : React Native pour l'application mobile et React classique pour l'interface web.

Nous avons réfléchi à la mise en place de l'architecture du projet. Pour la base de données, nous avons opté pour une structure relationnelle, plus normalisée et garantissant une bonne intégrité des données. C'est un modèle avec lequel nous sommes familiers et qui est donc facile à comprendre pour les membres de l'équipe. Nous avons opté pour une API, c'est-à-dire une interface permettant au Front-End de communiquer avec le back-end via des requêtes HTTP. Cette architecture a été pensée sur les principes REST, ce qui permet une gestion efficace des données et une communication fluide entre les différentes parties de l'application.

Le projet a alors été structuré de manière à séparer l'Api Back-End en relation avec la base de données, le Front-End web et le Front-End mobile.

Dans la partie API, nous avons à la racine les fichiers d'initialisation de l'environnement Python, puis les dossiers contenant les spécificités liées au recueil des données, ainsi que les dossiers associés au traitement des données, tels que les modèles, les vues, les filtres et les migrations. Django REST a été choisi pour sa réutilisabilité, car il permet de créer l'API de manière rapide et efficace tout en assurant une structure claire et modulaire.

Pour la partie web et mobile, nous avons opté pour l'Atomic Design, une méthodologie qui divise les composants en cinq niveaux hiérarchiques : atomes, molécules, organismes, templates et pages. Cette approche permet de créer des composants modulaires et réutilisables, facilitant ainsi une conception cohérente et évolutive pour tous les éléments de l'interface. Cette structure garantit que chaque élément de l'interface peut être facilement réutilisé et mis à jour sans perturber l'ensemble du projet.

2.2 Langages, frameworks et outils utilisés

Pour ce projet, nous avons sélectionné des technologies adaptées à nos besoins pour garantir une bonne modularité et une efficacité optimale. Voici les langages et les outils qui sont utilisés :

2.2.1 - Back-end avec Python et Django REST

Pour la partie Back-end, on utilise une base de données relationnelle MySQL. Celle-ci est alimentée et gérée par Django, un framework de Python.

Le Django a également été choisi car il permet de séparer efficacement le côté serveur qu'il gère avec le côté client.

2.2.2 - Front-End Web avec React TypeScript

Le Front-End est donc développé séparément et en deux différentes parties, le web et le mobile.

Nous avons opté pour React TypeScript pour la partie web, car nous maîtrisons déjà ce framework grâce à notre apprentissage de l'année passée.

2.2.3 - Front-End Mobile avec React Native

Et pour la partie mobile, nous avons choisi le React Native qui est également un framework TypeScript. Cela nous permettait de rester sur le langage TypeScript en Front-End afin de ne pas mélanger trop de langages par rapport au projet.

2.2.4 - Visual Studio Code

Pour développer ce projet, nous utilisons Visual Studio code, un éditeur de code qui démarre vite et fonctionne sans ralentir l'ordinateur, contrairement à d'autres outils plus lourds. VS CODE possède également différentes extensions utiles au développement Python et TypeScript, et à la gestion GitHub .

2.2.5 - Gestion de code avec GitHub

Afin d'avoir un dépôt distant pour une gestion collaborative du projet et un suivi de version efficace, nous avons opté pour GitHub .

GitHub facilite la collaboration au sein de l'équipe via des pull-requests, les branches et les commits. GitHub agit comme un outil centralisé où le code est sécurisé, accessible et facilement partageable, tout en simplifiant le travail en équipe.

2.2.6 - Tests avec Expo Go et Émulateurs

Nous utilisons Expo Go pour les tests utilisateurs et d'interface mobile, qui nous permet de lancer un serveur local et de simuler l'application sur nos téléphones personnels via un QR code.

2.2.7 - Suite Atlassian (Jira Software, Confluence)

Pour la gestion de projet, nous utilisons la suite Atlassian regroupant l'outil Jira, qui permet la gestion de la méthodologie Agile par sprint avec des tickets, de la planification de projet (Diagramme de Gantt, etc...) mais également d'avoir des rapports concrets sur l'avancée globale du projet.

Cette suite prend également en compte Confluence, un outil de centralisation des documents de l'équipe (notes de réunions, fiches techniques et rétrospectives de sprint).

Cet outil s'accompagne également d'un ensemble d'extension permettant par exemple d'avoir une vue sur la gestion collaborative GitHub ou encore sur les documents Google drive.

2.2.8 - Figma et Figma Dev

Pour le maquettage, nous utilisons Figma, un outil collaboratif de design d'interfaces et de prototypes. Nous le couplons avec l'onglet Dev de Figma, faite pour le développement, afin de pouvoir reprendre une partie du code (principalement l'esthétique) directement depuis la maquette. Cela nous fait gagner du temps sur le développement de l'application.

2.3. Explication des décisions technologiques

Maintenant que nous vous avons présenté les différents langages et outils utilisés, nous jugeons intéressant d'expliquer les raisonnements judicieux qui nous ont poussé à choisir ces technologies plutôt que d'autres.

2.2.1. - Back-end avec Python et Django REST

Nous connaissons bien le Python qui est un langage de programmation polyvalent, très compréhensible et facile à maintenir. Nous avons déjà appris le Python et donc nous sommes en mesure de nous concentrer davantage sur l'apprentissage du framework Django REST, spécialisé pour les API REST, ce qui est adapté pour les besoins du projet.

Django REST est un framework particulièrement adapté à ce projet, étant principalement orienté Back-end. Il met à disposition une grande variété d'outils préconfigurés, y compris pour les tests, l'envoi de données et la gestion des bases de données, ce qui rend le développement d'API RESTful robustes et sécurisées extrêmement facilité. Django offre une structure bien définie et des fonctionnalités intégrées, tout en assurant une bonne maintenance du code.

Il est très axé sur la sécurité et il utilise un ORM (Object-Relational Mapping), ce qui permet de manipuler facilement la base de données en utilisant des objets Python.

La base de données MySQL lui est associée en raison de sa comptabilité avec l'ORM de Django. De plus, nos connaissances de BUT nous permettent de mieux connaître le requêtage en bases de données relationnelles et donc de mieux sécuriser les connexions et requêtes vers la base de données.

Seul un dump modifié de la base de données sera donné, la base de données en ligne sera à refaire une fois la présentation effectuée.

2.2.2. - Front-End Web avec React TypeScript

Le React TypeScript a été choisi pour la partie web en raison de son intégration fluide avec le Back-End Python et de sa bonne communication avec les API RESTful (avec le module Axios). TypeScript apporte d'ailleurs une vérification statique des types par rapport au JavaScript, ce qui réduit les erreurs et améliore la collaboration entre le Front-End et le Back-End.

2.2.3. - Front-End Mobile avec React Native

Le React Native pour la partie Front-End mobile apporte des avantages similaires au React utilisé pour le web. Sa plus grosse différence avec le React est que c'est un framework TypeScript orienté vers le mobile et donc permet le développement à la fois IOS et Android. Nous l'avons donc choisi pour cibler la plupart des téléphones utilisés par les utilisateurs de l'application mobile.

Ce langage nécessite cependant un apprentissage de la part de l'équipe étant donné que nous ne l'avons jamais utilisé auparavant.

2.2.4. - Visual Studio Code

Visual Studio Code est notre éditeur de code favori pour travailler avec Python et TypeScript en raison de sa prise en charge native de ces langages, de son large éventail d'extensions pour personnaliser l'environnement de développement, ce qui rend le développement plus rapide. Son interface intuitive facilite la gestion des fichiers et des projets, tandis que son intégration avec GitHub simplifie la gestion du code collaboratif.

En cours et en apprentissage, il nous a été recommandé pour TypeScript d'utiliser VSCode.

2.2.5. - Gestion de code avec GitHub

GitHub est un outil de gestion collaborative qui a été adopté par notre groupe étant donné sa simplicité d'utilisation et sa documentation très complète. Nous avons déjà fait plusieurs projets dessus donc l'apprentissage de cet outil ne requiert aucune formation pour l'équipe.

GitHub est également un choix incontournable pour le développement grâce à ses outils de collaboration avancés, tels que les pull-requests (voir Partie 5 - Tests et validation) pour la révision du code et la gestion des branches.

2.2.6. - Tests avec Expo Go et Émulateurs

L'approche des tests utilisateurs par émulateurs garantit une couverture complète des scénarios d'utilisation, tout en détectant et corrigeant rapidement les éventuels problèmes spécifiques à chaque plateforme. La vérification du code est très rapide et facile grâce à la connexion internet entre l'ordinateur qui ouvre le serveur local et le téléphone, ce qui rend Expo Go extrêmement simple.

2.2.7. - Suite Atlassian (Jira Software, Confluence)

Pour la gestion de projet, la suite Atlassian regroupe une bonne partie de la gestion de projet grâce à la centralisation des outils (Github Pull Requests ou encore Google Drive). De plus, Atlassian offre des fonctionnalités dédiées à la gestion de projet, avec Jira Software pour le suivi des tâches et des sprints (méthode Agile), ainsi que des outils pour la gestion de la documentation (Confluence), notamment pour organiser et consigner les informations provenant des réunions.

2.2.8. - Figma et Figma Dev

Figma a été choisi par notre équipe pour le prototypage car c'est un outil intuitif et très simple à utiliser. Il permet de prototyper l'application en ligne et de concevoir rapidement des designs et des prototypes d'applications détaillés.

Nous utilisons la version de développement afin d'avoir le code des composants directement au format HTML/CSS. Cela permet alors de réduire le temps de développement une fois la maquette construite.

Nous connaissons bien Figma donc hormis quelques apprentissages, nous maîtrisons bien l'outil.

2.2.9. - Docker

L'outil Docker est utilisé pour encapsuler notre API Django REST dans un environnement isolé afin de simuler un environnement de production pendant la phase d'intégration des applications à l'API. Cela permet d'éviter les problèmes de compatibilité entre les environnements de développement des membres de l'équipe notamment pour l'application portable.

Cette instance ne sera plus disponible une fois la présentation effectuée.

Le lien vers le document relatant de l'architecture technique et des choix réalisé par notre équipe est disponible en annexe 2 - Architecture et choix techniques

2.4. Verrous techniques du développement

Nous développons encore notre POC actuellement mais nous avons déjà fait face à quelques verrous techniques durant les réflexions portées sur certaines fonctionnalités ou encore le développements de celles dont nous allons vous faire part. Cette liste de verrous sera amenée à évoluer au cours du projet.

2.4.1 : Verrou concernant l'ADE

Tout d'abord, il n'existe pas d'API nous permettant d'accéder en lecture et en écriture à la base de données qui alimente l'ADE. Donc pour la fonctionnalité liée à la réservation de salle, nous sommes obligés d'utiliser une solution alternative à savoir répertorier les réservations de salles à travers notre base de données. Mais cette solution peut engendrer des conflits par la suite avec la modifications d'horaire des salles dans l'ADE.

Etant donné que nous ne pouvons pas avoir accès à l'ADE en temps réel, nous ne pouvons pas notifier et afficher les changements de cours auprès des utilisateurs.

Il faudrait alors optimiser les interactions avec le logiciel ADE pour permettre une meilleure gestion des emplois du temps. Pour ce faire, voici des améliorations qui peuvent être possibles:

- Mise en place d'une API bidirectionnelle pour automatiser la lecture et l'écriture des données (emplois du temps, événements).
- Génération de liens dynamiques permettant la visualisation interactive des emplois du temps, filtrés par intervalle ou d'autres critères (promotion, salle, cours spécifique).
- Ajout d'une fonctionnalité de synchronisation automatique avec les outils tiers (ex. : Google Calendar, Outlook).

2.4.2 : Verrou concernant le paiement Izly

Ensuite, pour la fonctionnalité de réservation de repas en Click and collect, nous ne pouvons pas intégrer un paiement via Izly qui est le seul moyen de payer au CROUS actuellement. Cette fonctionnalité peut se faire, en accord avec le CROUS, mais a été jugée inutile dans le cadre d'un POC où l'on s'intéresse avant tout à mettre en évidence les concepts associés au projet et non à développer une version complète et fonctionnelle de celui-ci.

Nous implémentons cependant une page de paiement en Front-End afin de montrer un visuel en lien avec le cas d'utilisation du paiement de la réservation de repas.

L'application finale post-POC devra faire l'objet d'une modification relative à l'implémentation des paiements. Elle comprendra notamment la simplification et la sécurisation des transactions liées aux services de réservation en Click and collect en développant :

- Une intégration de l'API Izly dans le système pour permettre le paiement direct via une interface utilisateur simplifiée.
- Un ajout de fonctionnalités comme le suivi des paiements pour les étudiants et le personnel administratif.
- Une implémentation d'un module de gestion des autorisations pour limiter ou prioriser certains types de paiements en fonction des rôles ou des droits des utilisateurs.

2.4.3 : Verrou concernant les droits

Enfin, plusieurs questions sont survenues concernant les droits d'accès et la gestion des données. En effet, les professeurs seraient en mesure d'effectuer des modifications critiques sur les données telles que des suppressions de comptes.

Il faudrait donc offrir un contrôle granulaire des droits utilisateurs, notamment pour les professeurs et les administrateurs via :

- la création d'un système de rôles et permissions hiérarchisé :
 - Professeurs : droits restreints pour éviter des modifications critiques (ex. : suppression ou annulation de réservations sans justification).
 - Administrateurs : accès complet pour gérer les utilisateurs, les réservations, et les configurations système.
- l'ajout d'une fonctionnalité de journalisation pour suivre toutes les modifications effectuées (traçabilité des actions).

Les professeurs ayant les droits d'administration sur le calendrier de réservations de salles et box se doivent d'être bienveillants. Il est chargé par l'administrateur d'octroyer ces droits avec précaution sur le POC.

2.4.4 : Verrou concernant la réalité augmentée

Pour la réalité augmentée, nous savions lorsque nous avons pris le projet que nous n'avions pas les compétences pour le faire en intégralité.

Nous avons basé nos recherches sur des solutions de réalité augmentée pour le POC comme l'API Web XR mais n'avons rien trouvé qui soit suffisamment optimal pour apprendre la technologie et l'appliquer au sein de notre POC.

Nous avons fait prendre conscience de ce verrou à la MOA qui l'a très bien accepté et à valoriser notre approche qui consiste alors en un scan de QR code suivi d'une photo panoramique avec pour but de cibler la zone vers où l'on doit se diriger et afficher l'itinéraire marqué par une flèche.

2.4.5 : Verrou concernant le NFC pour les frigos connectés

Les frigos connectés devaient de base être gérés par scanner mais par soucis d'équipements, nous n'avons pas pu gérer l'IoT des frigos connectés directement dans l'application.

En alternative, nous avons opté pour la gestion des réservations de repas en générant un QR code classique, permettant une solution simple et fonctionnelle.

2.4.6 : Verrou concernant l'absence de matériel / personnel lors du POC

Que ce soit pour les réservations de box acoustiques ou la gestion des frigos connectés, ne pas avoir de matériels en fonction nous a bloqué dans certaines démarches.

Pour la réservation des box acoustiques, nous avons trouvé une alternative par les salles qui présentent les mêmes caractéristiques à savoir, un nombre de places, un emplacement ou encore des horaires pour les disponibilités.

2.4.7 : Verrou concernant les serveurs mis à disposition

L'un des défis majeurs du projet a été l'absence de serveurs préconfigurés pour héberger nos services. Nous avons eu accès à un serveur tiers mais nous devons le configurer afin d'héberger notre base de données et d'avoir les outils nécessaires pour la sécurisation. Ce verrou nécessite donc un temps supplémentaire pour assurer un environnement stable et fonctionnel pour le projet.

Une mise à disposition de serveur préconfiguré par l'université pour ce type de projet serait un plus qui aiderait énormément à l'avenir.

3. Conception et prototypage

Après avoir défini les besoins et identifié les fonctionnalités clés du projet, nous avons entamé la phase de conception et prototypage. Pour structurer le projet, il faut traduire les exigences fonctionnelles en éléments concrets et exploitables pour le développement.

Nous avons d'abord repris et analysé l'ancienne maquette qui était malheureusement inexploitable en code avec l'outil Figma Dev, et manquait d'éléments concrets comme la barre de navigation pour s'orienter sur l'application. La refonte de la maquette Figma a permis de visualiser l'interface utilisateur et de garantir une expérience fluide et intuitive. Elle a été approuvée par la MOA et les responsables du projet.

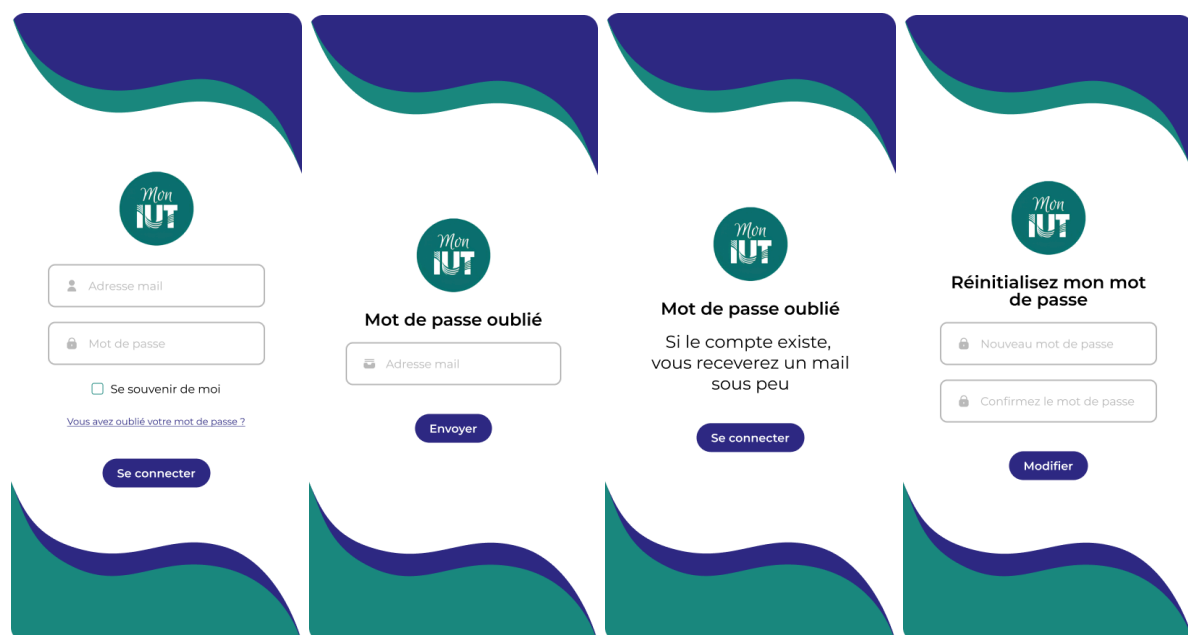
Ensuite, nous avons travaillé sur l'architecture du système, définissant la répartition entre les différentes parties de l'application (Front-End et back-end).

Enfin, nous avons élaboré la base de données en structurant les données et leurs relations, en prenant en compte les besoins déterminés dans le cahier des charges.

3.1 Maquettes de l'application

Application Mobile

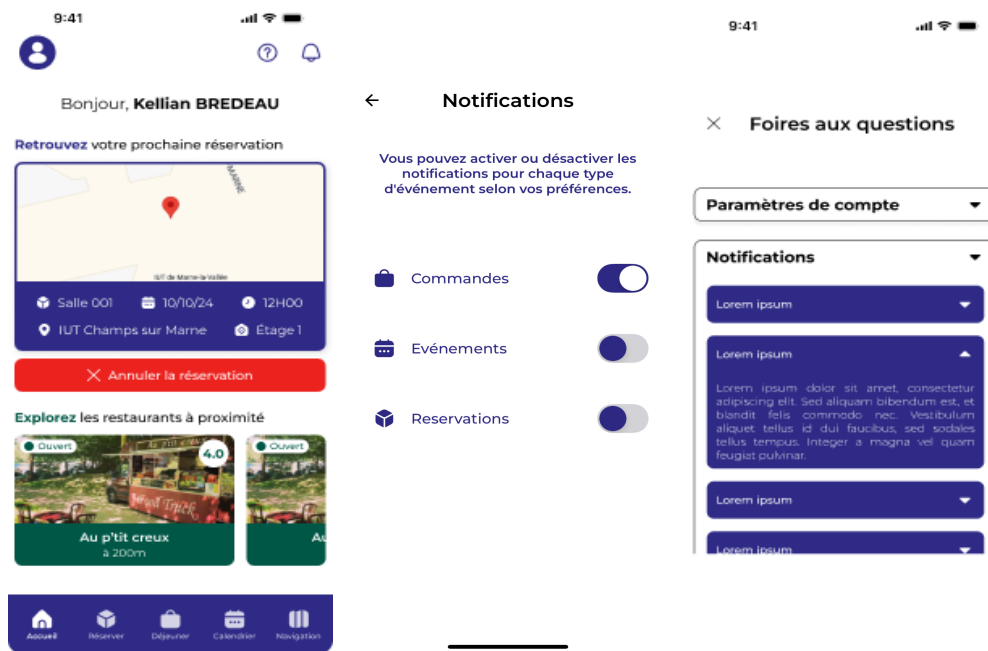
Connexion à l'application mobile



Ecran d'exemple n°1-4: Pages de connexion

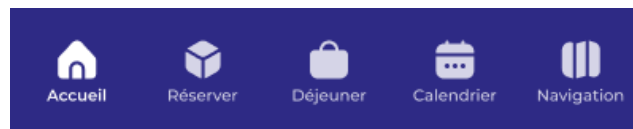
La connexion sur l'application (écran 1) se fait avec l'adresse mail du compte et son mot de passe. En cas d'oubli du mot de passe (écran 2), il est possible d'envoyer un mail (écran 3) sur la boîte de réception pour pouvoir réinitialiser le mot de passe (écran 4). Pour ne pas rentrer les identifiants à chaque ouverture de l'application, nous pouvons retenir les données en cache.

Accueil, Notification, FAQ et Navigation dans l'application



Ecran d'exemple n°5-7: Accueil, Notification et FAQ

L'écran d'accueil (écran 5) permet d'accéder à l'ensemble des fonctionnalités de l'application. Il affiche la dernière réservation à venir avec ses informations, ainsi que la liste des lieux de restauration à proximité, accessibles pour le click and collect. Nous avons également mis en place le réglage des notifications à disposition de l'utilisateur (écran 6) et la foire aux questions de l'établissement (écran 7) qui est contrôlable par l'administrateur.



Ecran d'exemple n°8: Barre de Navigation

La barre de navigation (écran 8) est accessible depuis la plupart des écrans et regroupe l'accès aux fonctionnalités principales du POC pour les étudiants :

- Accueil : La page actuelle.
- Réserver : La page de réservation des box acoustiques.
- Déjeuner : La page pour le click and collect sur les frigos connectés et les restaurants.
- Calendrier : Le calendrier affichant les réservations et l'emploi du temps lié à ADE.
- Carte : Le plan de l'établissement avec les points d'intérêt.



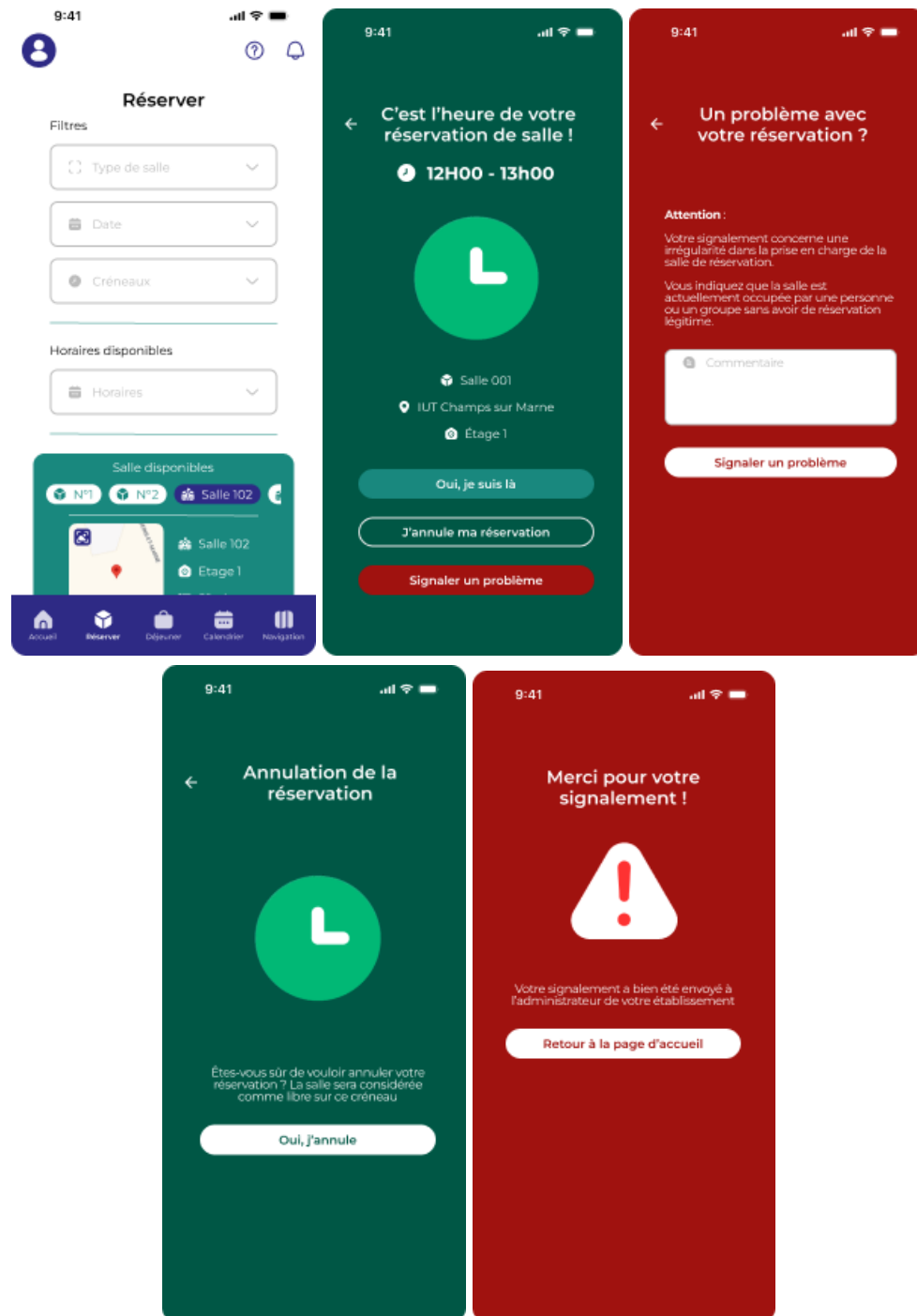
Ecran d'exemple n°9: Header de la page principale

La barre en haut (écran 9) permet d'accéder à des fonctionnalités secondaires du projet :

ChillZone - Projet BUT Informatique 3

- Le compte : Toutes les options de modification liées aux informations de l'étudiant sont regroupées dans cette section.
- La foire aux questions : Suite à la demande d'ajout de cette fonctionnalité, elle regroupe des questions fréquemment posées ainsi que leurs réponses, classées par catégories.
- Les notifications : Cette section présente la partie réglage des notifications que le projet peut envoyer selon les types de notifications.

Réservation d'une salle

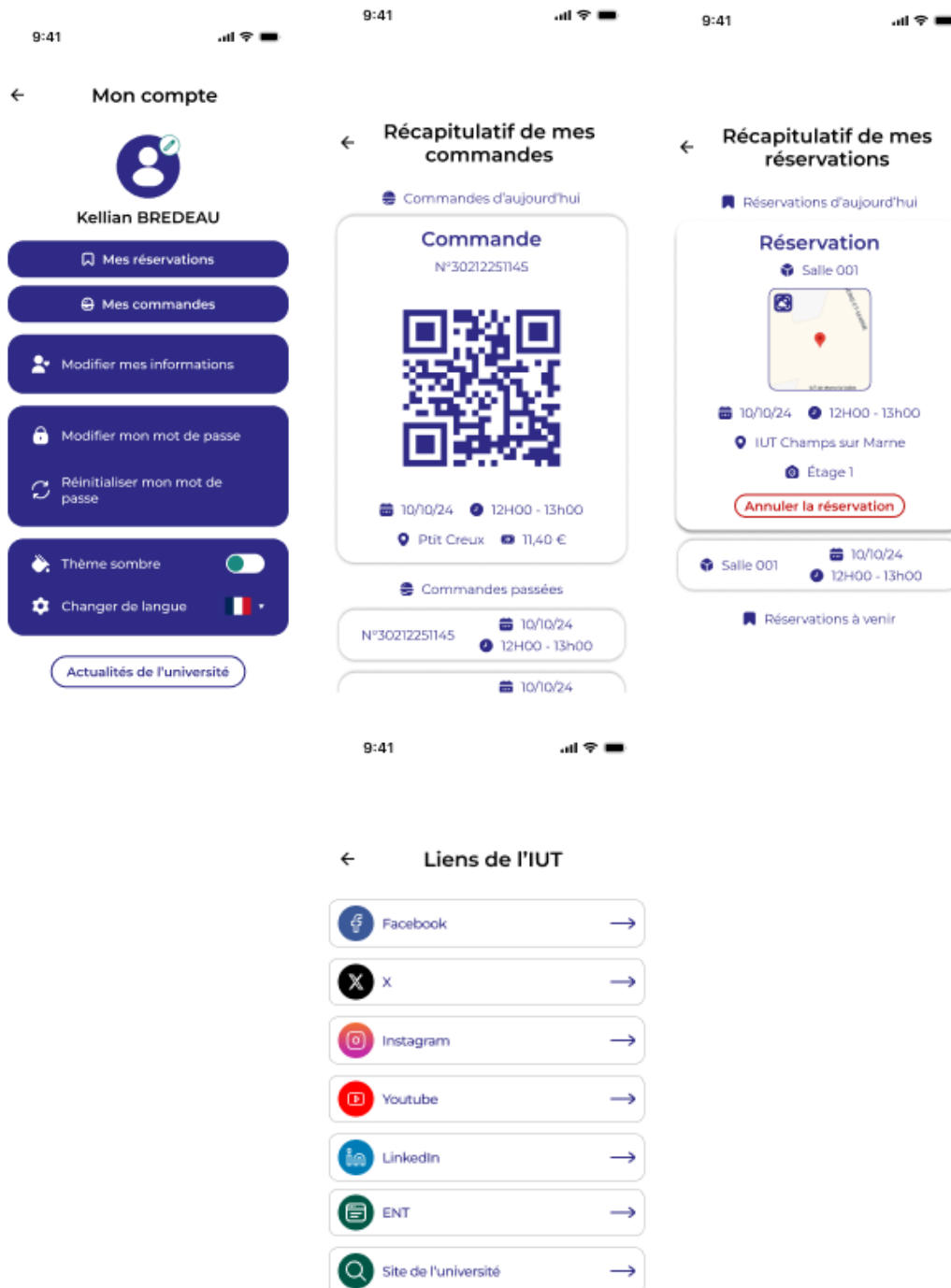


Ecran d'exemple n°10-15: Pages de réservation de salle

Sur l'écran réservation (écran 10), on peut filtrer les salles / box acoustiques disponibles par le type de salle désiré, la date et le créneaux (30min, 1h, 2h) demandés par l'utilisateur. Selon les sélections faites, l'application propose des horaires et des salles disponibles à la réservation.

A l'heure de la réservation, un écran (écran 11) apparaît pour confirmer la prise en main de la salle ; l'utilisateur peut alors confirmer, annuler (écran 13) ou signaler un problème de salle (écran 12 et 14) à l'administrateur de l'établissement.

Compte Utilisateur



ChillZone - Projet BUT Informatique 3

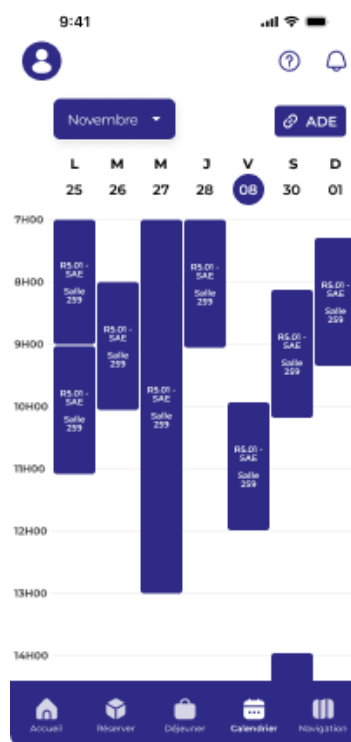
Ecran d'exemple n°15-18: Pages de compte utilisateur

Sur l'espace compte (*écran 15*), l'utilisateur peut consulter et modifier ses informations personnelles. Il a la possibilité de changer la langue de l'application, disponible en français et en anglais.

Il peut également accéder aux liens des réseaux sociaux de son établissement pour se tenir informé des dernières actualités (*écran 18*).

De plus, l'utilisateur peut visualiser un récapitulatif de ses réservations de box acoustiques (*écran 16*) ainsi que de ses commandes passées en click and collect (*écran 17*).

Calendrier

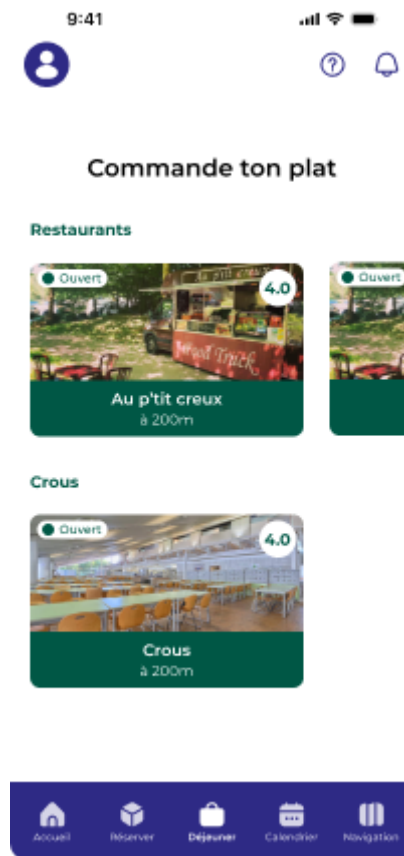


Ecran d'exemple n°19: Calendrier

Sur la page calendrier (*écran 19*), l'utilisateur peut voir ses cours via la synchronisation ADE ainsi que ses réservations de salles via le POC. Tous les créneaux sont cliquables pour plus d'information à leur sujet.

Avec le bouton "ADE", l'utilisateur peut mettre son lien d'exportation ICAL pour l'afficher correctement dans notre application.

Click and Collect



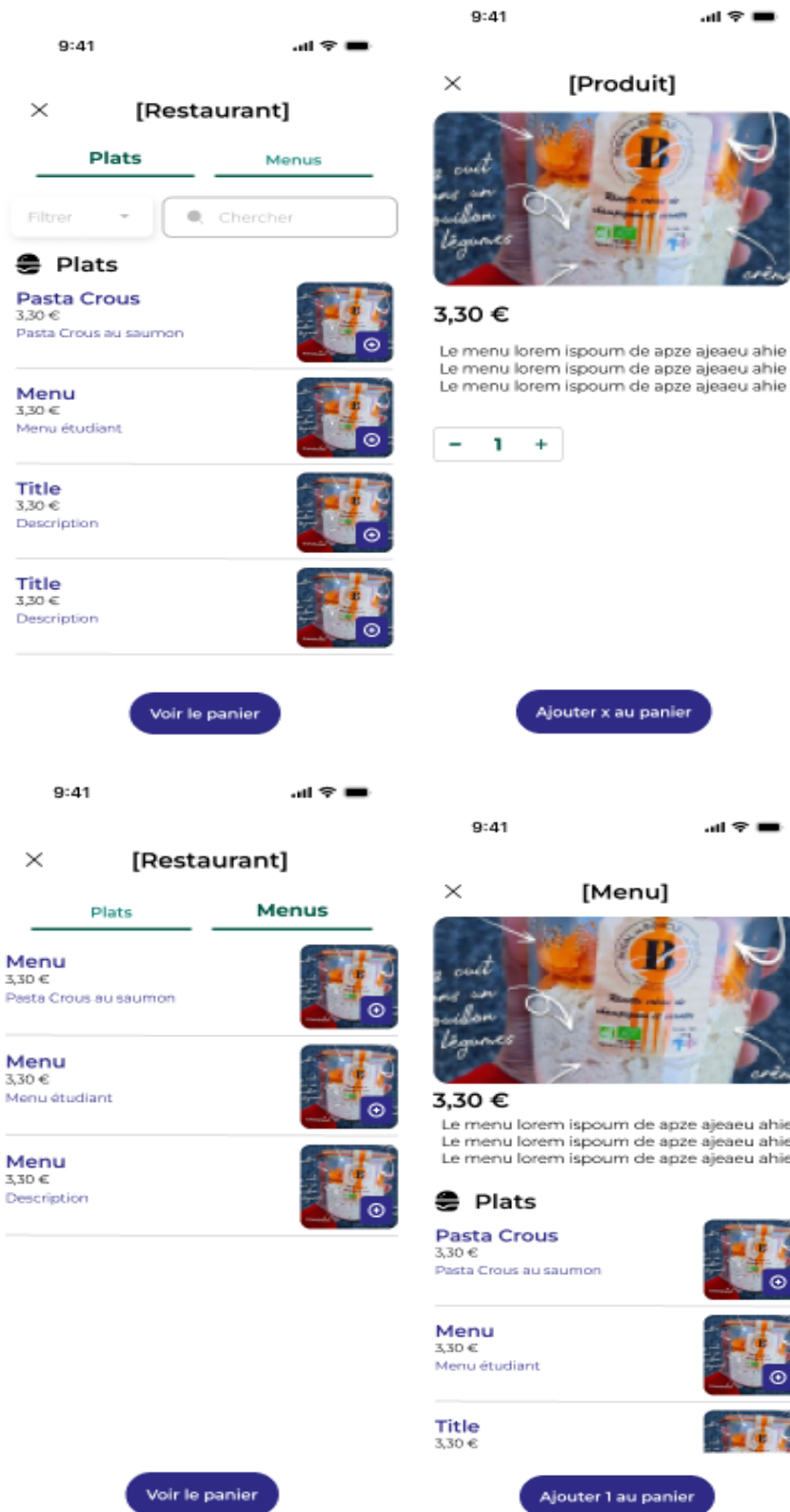
Ecran d'exemple n°20: Page de réservation de repas

Dans l'espace click and collect (écran 20), l'utilisateur peut d'abord voir les différents restaurants, indiqués comme ouverts ou fermés. En cliquant sur un restaurant ou un frigo, il entame le processus de commande click and collect.

Ensuite, l'utilisateur peut ajouter des menus ou des plats individuels en plusieurs quantités dans son panier. Il a la possibilité de filtrer les options par catégories ou de rechercher des éléments par nom.

Une fois sa sélection effectuée, l'utilisateur peut valider ou modifier son panier, puis procéder au paiement. Après le paiement, un QR code de récupération de commande lui est généré, qu'il pourra utiliser pour récupérer sa commande soit à la borne du frigo, soit en caisse auprès du restaurateur.

ChillZone - Projet BUT Informatique 3



Ecran d'exemple n°21-24: Pages d'affichage des plats et des menus

Voici les écrans de sélection de plats (écran 21) et menus (écran 23) disponibles pour un restaurant choisi. Si on appuie sur un plat (écran 22) on peut choisir la quantité à ajouter au panier. Si on appuie sur un menu (écran 24) on peut choisir les plats contenus dans le menu.

ChillZone - Projet BUT Informatique 3

9:41

←

Panier

Commande au p'tit creux



Votre créneau de collecte

Standard

Prévu à

Votre commande

Muffin chocolat

Menu First

Sandwich Parisien

Fanta Orange

Donut Daims

3,30 €

3 €

TOTAL

6.30 €

PAYER MAINTENANT

9:41

←

Paieement

Carte de crédit / débit

Tous les champs doivent être remplis

Numéro de la carte

Nom de la carte

Date d'expiration

Code de sécurité

Format MM/YY

Les 3 chiffres au dos de la carte

Payer 6.30€

En procédant au paiement de votre repas, vous acceptez de renoncer à votre droit de rétractation conformément aux dispositions légales applicables aux biens ou services immédiatement consommables ou personnalisés.

9:41

×

Bon appétit !

Votre commande n°106501530150 a été confirmée



TÉLÉCHARGER LE QR CODE

C'est prêt ! Rendez-vous au point de collecte choisi et scannez ce QR Code pour savourer votre plat !

Écran d'exemple n°25-27: Pages de réservations de repas

Voici les écrans pour la confirmation du panier (écran 25), du paiement (écran 26) et de la finalisation de la commande (écran 27) où on reçoit le qr code à présenter lors de la récupération de la commande.

Application Web

Connexion à l'interface web back-office



Mot de passe oublié

Si le compte existe,
vous recevrez un mail sous
peu



Mot de passe oublié

Si le compte existe,
vous recevrez un mail sous
peu

Mon iUT

Réinitialisez votre mot de passe

Nouveau mot de passe

Confirmez le mot de passe

[Modifier](#)

Mon iUT

Connexion au back-office

[Je veux m'inscrire en tant que restaurateur](#)

Adresse mail

Mot de passe

☐ Se souvenir de moi

[J'ai oublié mon mot de passe](#)

[Se connecter](#)

Ecran d'exemple n°30-33 : Pages de connexion des administrateurs sur PC

La connexion web (écran 30) se fait avec l'adresse mail du compte et son mot de passe. En cas d'oubli du mot de passe (écran 31), il est possible d'envoyer un mail sur la boîte de réception (écran 32) pour pouvoir réinitialiser le mot de passe (écran 33). Pour ne pas rentrer les identifiants à chaque ouverture de l'application, nous pouvons retenir les données en cache.

Il est possible de faire une demande d'inscription pour l'utilisateur restaurateur.

Inscription en tant que restaurateur

Mon iUT

Demande d'inscription restaurateur - Restaurant

Nom du lieu de restauration

Description

Heure ouverture

Heure fermeture

Emplacement

IMAGE

☐ Frigo connecté ☐ Restaurant

Telephone

Etablissement

[Retour](#) [Confirmer l'inscription](#)

Mon iUT

Demande d'inscription envoyée à l'administrateur

Votre demande a été bien envoyée à l'administrateur de l'établissement concerné par votre demande. Vous recevrez un mail d'ici peu

[Retourner à la page de connexion](#)

Ecran d'exemple n°34-36: Pages de connexion du restaurateur sur PC

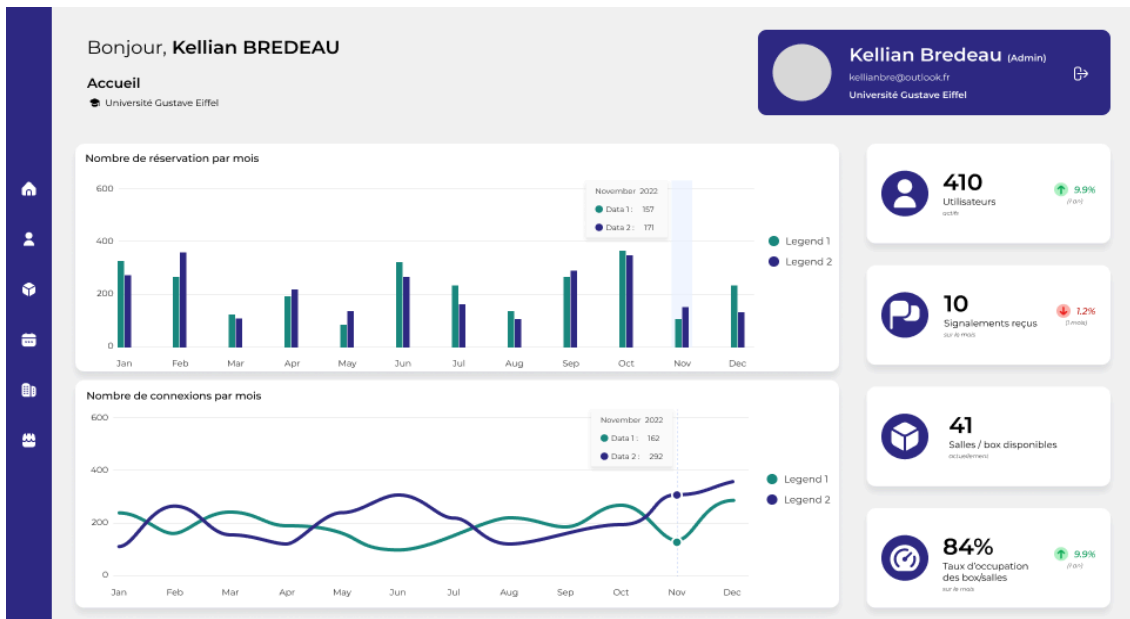
Lors de l'inscription d'un utilisateur, il y a deux formulaires à remplir :

- Formulaire de compte client (écran 34) : L'utilisateur doit fournir son email, nom, prénom, et numéro de téléphone.
- Formulaire de restaurant (écran 35): L'utilisateur renseigne les informations nécessaires relatives à son restaurant (nom, emplacement, horaires d'ouvertures).

Les informations soumises sont ensuite en attente de validation par le super administrateur avant que l'inscription ne soit finalisée (écran 36).

ChillZone - Projet BUT Informatique 3

Ecrans Administrateur d'Établissement



Bonjour, **Kellian BREDEAU**

Comptes
Université Gustave Eiffel

Kellian Bredeau (Admin)
kellianbre@outlook.fr
Université Gustave Eiffel

Chercher dans la table Columns Filters Density Export Ajouter un compte utilisateur

Nom Prénom	Adresse mail	Rôle	Établissement	Nombre de réservations faites	Bloquer les réservations	Supprimer le compte
Dupont Olivier	dudu@univ-eiffel.fr	Etudiant	IUT - Antenne Marne la vallée	15	☐	
Dupont Olivier	dudu@univ-eiffel.fr	Etudiant	IUT - Antenne Marne la vallée	13	☒	
Dupont Olivier	dudu@univ-eiffel.fr	Professeur	IUT - Antenne Marne la vallée	12	☒	
Dupont Olivier	dudu@univ-eiffel.fr	Etudiant	IUT - Antenne Marne la vallée	11	☒	
Dupont Olivier	dudu@univ-eiffel.fr	Professeur	IUT - Antenne Marne la vallée	11	☒	
Dupont Olivier	dudu@univ-eiffel.fr	Etudiant	IUT - Antenne Marne la vallée	11	☐	
Dupont Olivier	dudu@univ-eiffel.fr	Etudiant	IUT - Antenne Marne la vallée	8	☒	
Dupont Olivier	dudu@univ-eiffel.fr	Etudiant	IUT - Antenne Marne la vallée	8	☐	
Dupont Olivier	dudu@univ-eiffel.fr	Etudiant	IUT - Antenne Marne la vallée	7	☐	
Dupont Olivier	dudu@univ-eiffel.fr	Professeur	IUT - Antenne Marne la vallée	3	☒	
Dupont Olivier	dudu@univ-eiffel.fr	Etudiant	IUT - Antenne Marne la vallée	3	☐	
Dupont Olivier	dudu@univ-eiffel.fr	Etudiant	IUT - Antenne Marne la vallée	2	☐	
Dupont Olivier	dudu@univ-eiffel.fr	Etudiant	IUT - Antenne Marne la vallée	2	☐	
Dupont Olivier	dudu@univ-eiffel.fr	Professeur	IUT - Antenne Marne la vallée	1	☒	

15 of 10

ChillZone - Projet BUT Informatique 3

Salles
Université Gustave Eiffel

Kellian Bredeau (Admin)
kellianbre@outlook.fr
Université Gustave Eiffel

ColumnsFiltersDensityExportAjouter une salle

Nom de la salle	Description de la salle	Capacité de la salle	Etage	Etablissement	La salle est libre actuellement ?	Modifier
Salle 023	Salle avec des chaises et des bureaux et un tableau blanc et un vidéoprojecteur	28	0	IUT - Antenne Marne la vallée	☒	Modifier
Salle 024	Salle avec des chaises et des bureaux et un tableau blanc	32	0	IUT Antenne Marne la vallée	☒	Modifier
Salle 028	Salle avec des chaises et des bureaux et un tableau blanc	28	0	IUT Antenne Marne la vallée	☒	Modifier
Box 101	Box acoustique pour des réunions	4	1	IUT Antenne Marne la vallée	☒	Modifier
Box 102	Box acoustique pour des réunions	4	1	IUT Antenne Marne la vallée	☒	Modifier
Box 103	Box acoustique pour des réunions	8	1	IUT Antenne Marne la vallée	☒	Modifier
IUC Blaise Pascal	Grande salle pour les cours magistraux et les présentations	81	2	IUT Antenne Marne la vallée	☒	Modifier
Box 201	Box acoustique pour des réunions	4	2	IUT Antenne Marne la vallée	☒	Modifier
Box 202	Box acoustique pour des réunions	2	2	IUT Antenne Marne la vallée	☒	Modifier
Box 203	Box acoustique pour des réunions	2	2	IUT Antenne Marne la vallée	☒	Modifier
Salle 223	Salle avec des chaises et des bureaux et un tableau blanc	23	1	IUT Antenne Marne la vallée	☒	Modifier
Salle 225	Salle avec des chaises et des bureaux et un tableau blanc et un vidéoprojecteur	28	1	IUT Antenne Marne la vallée	☒	Modifier
Salle 224	Salle avec des chaises et des bureaux et un tableau blanc	28	1	IUT Antenne Marne la vallée	☒	Modifier
Salle 265	Salle avec des chaises et des bureaux et un tableau blanc	32	1	IUT Antenne Marne la vallée	☒	Modifier

14 of 10

Réervations & Signalements
Université Gustave Eiffel

Kellian Bredeau (Admin)
kellianbre@outlook.fr
Université Gustave Eiffel

Tableau des réservations

Tableau des signalements

ColumnsFiltersDensityExport

ID réservation	Date	Début	Fin	Salle/Box	Status	Arrivé ?
01225151	10/12/2024	14:00	15:00	Salle 024	Annulé	☒
0165648	10/12/2024	15:00	16:00	Salle 024	Réservé	☒
16541616	10/12/2024	14:30	15:00	Box 101	Réservé	☒
25163182	10/12/2024	11:00	11:30	Box 101	Fin	☒
06616212	10/12/2024	12:30	14:30	Box 101	Fin	☒
0548694	10/12/2024	16:00	16:30	Box 101	Fin	☒
25151555	10/12/2024	13:00	14:00	Salle 028	Annulé	☒
25841515	12/12/2024	13:00	14:00	IUC Blaise Pascal	Fin	☒
6525655	12/12/2024	13:00	14:00	Box 101	Fin	☒
9656595	12/12/2024	13:00	14:00	Box 101	Fin	☒
11111111	12/12/2024	13:00	14:00	Salle 024	Fin	☒
6565656	12/12/2024	13:00	14:00	Salle 024	Fin	☒
45121212	12/12/2024	13:00	14:00	Salle 024	Fin	☒
3233333	12/12/2024	13:00	14:00	Salle 024	Fin	☒

14 of 10

ColumnsFiltersDensityExport

14 of 10

Plans & Informations de l'Etablissement
Université Gustave Eiffel

Kellian Bredeau (Admin)
kellianbre@outlook.fr
Université Gustave Eiffel

Etages de l'établissement
RDC
Etage 1
Etage 2
Ajouter un étage

Information du point sélectionné
Veuillez sélectionner un point sur la carte

Etage 1
Changer de plan

Informations de l'établissement
Nom
Adresse
Ville
Code Postal
Téléphone
Adresse mail
Sauvegarder

ChillZone - Projet BUT Informatique 3

Bonjour, Admin Test

Gestion de la Foire aux questions

Université Gustave Eiffel

Admin Test (Administrateur)

admin@univ-eiffel.fr

Université Gustave Eiffel

Chercher dans la table

COLUMNS

FILTERS

DENSITY

EXPORT

AJOUTER UNE QUESTION

ID	Catégorie	Question	Réponse	Modifier	Supprimer
1	Paramètres de compte	Comment modifier mes informations personnelles (nom, prénom, etc.) ?	Cliquez sur votre photo de profil en haut à gauche sur la page d'Accueil puis cliquez sur "Modifier mes informations".		
11	Paramètres de compte	Comment modifier mon mot de passe ?	Cliquez sur votre photo de profil en haut à gauche sur la page d'Accueil puis cliquez sur "Modifier mon mot de passe".		
12	Paramètres de compte	Que faire si j'ai oublié mon mot de passe ?	Si vous êtes déjà connecté, cliquez sur votre photo de profil puis cliquez sur "Réinitialiser mon mot de passe" et rentrez votre adresse mail de votre compte. Si vous n'êtes pas connecté, cliquez sur "Mot de passe oublié" et suivez les instructions.		
14	Données et confidentialité	Comment mes données personnelles sont-elles protégées ?	L'accès à vos données personnelles sont protégées par l'API avec les systèmes de sessions.		
15	Données et confidentialité	L'application partage-t-elle mes données avec des tiers ?	Non. Sauf si vous considérez votre téléphone comme un "tiers", parce qu'il en sait beaucoup sur vous.		
16	Click & Collect (repas)	Comment passer une commande en Click & Collect ?	Cliquez sur "Déjeuner", sélectionnez le restaurant chez qui vous souhaitez passer commande puis faites votre panier !		
17	Click & Collect (repas)	Quels types de repas sont disponibles à la commande ?	Une multitude ! Cela dépend des restaurants affiliés à votre établissement.		
18	Click & Collect (repas)	Puis-je commander pour plusieurs personnes en même temps ?	Oui, la commande peut contenir plusieurs menus ou plats en fonction de vos envies ou besoins.		
19	Click & Collect (repas)	Puis-je choisir un bon plat en fonction de mon régime alimentaire ?	Oui, il est possible de choisir un plat végétarien, sans gluten, etc.		

Rows per page: 100 1-16 of 16

Réseaux de l'établissement

AJOUTER

https://www.facebook.com/Universit

https://www.instagram.com/universit

https://www.linkedin.com/company/h

https://www.youtube.com/c/Universit

https://www.univ-gustave-eiffel.fr

https://intranet-edu.univ-eiffel.fr/acce

https://bsky.app/profile/univeiffel.bsl

Bonjour, Kellian BREDEAU

Lieux de restauration affiliés

Université Gustave Eiffel

Kellian Bredeau (Admin)

kellianbre@outlook.fr

Université Gustave Eiffel

Chercher dans la table

COLUMNS

FILTERS

DENSITY

EXPORT

Tableau des lieux de restauration affiliés à l'établissement

Nom du lieu de restauration	Type de lieu de restauration	Emplacement du lieu de restauration	Adresse mail du lieu de restauration	Ouvert actuellement	Supprimer le lien entre le lieu et l'établissement
Frigo 401	Frigo connecté	IUT - Antenne Marne la vallée	dudu@univ-eiffel.fr	OUI	
Frigo 402	Frigo connecté	IUT - Antenne Marne la vallée	dudu@univ-eiffel.fr	OUI	
Restaurant 101	Restaurant	IUT - Antenne Marne la vallée	dudu@univ-eiffel.fr	NON	
Restaurant 101	Restaurant	IUT - Antenne Marne la vallée	dudu@univ-eiffel.fr	NON	
Restaurant 101	Restaurant	IUT - Antenne Marne la vallée	dudu@univ-eiffel.fr	NON	

1-5 of 10

Tableau de demandes d'affiliation des lieux de restauration à l'établissement

Nom du lieu de restauration	Type de lieu de restauration	Emplacement du lieu de restauration	Adresse mail du lieu de restauration	Accepter l'affiliation	Refuser l'affiliation
Frigo 401	Frigo connecté	IUT - Antenne Marne la vallée	dudu@univ-eiffel.fr		
Frigo 402	Frigo connecté	IUT - Antenne Marne la vallée	dudu@univ-eiffel.fr		
Restaurant 101	Restaurant	IUT - Antenne Marne la vallée	dudu@univ-eiffel.fr		
Restaurant 101	Restaurant	IUT - Antenne Marne la vallée	dudu@univ-eiffel.fr		

1-5 of 10

Ecran d'exemple n°37-43: Pages des fonctionnalités de l'administrateur sur PC

Ces écrans regroupent les fonctionnalités pour l'administrateur d'établissement :

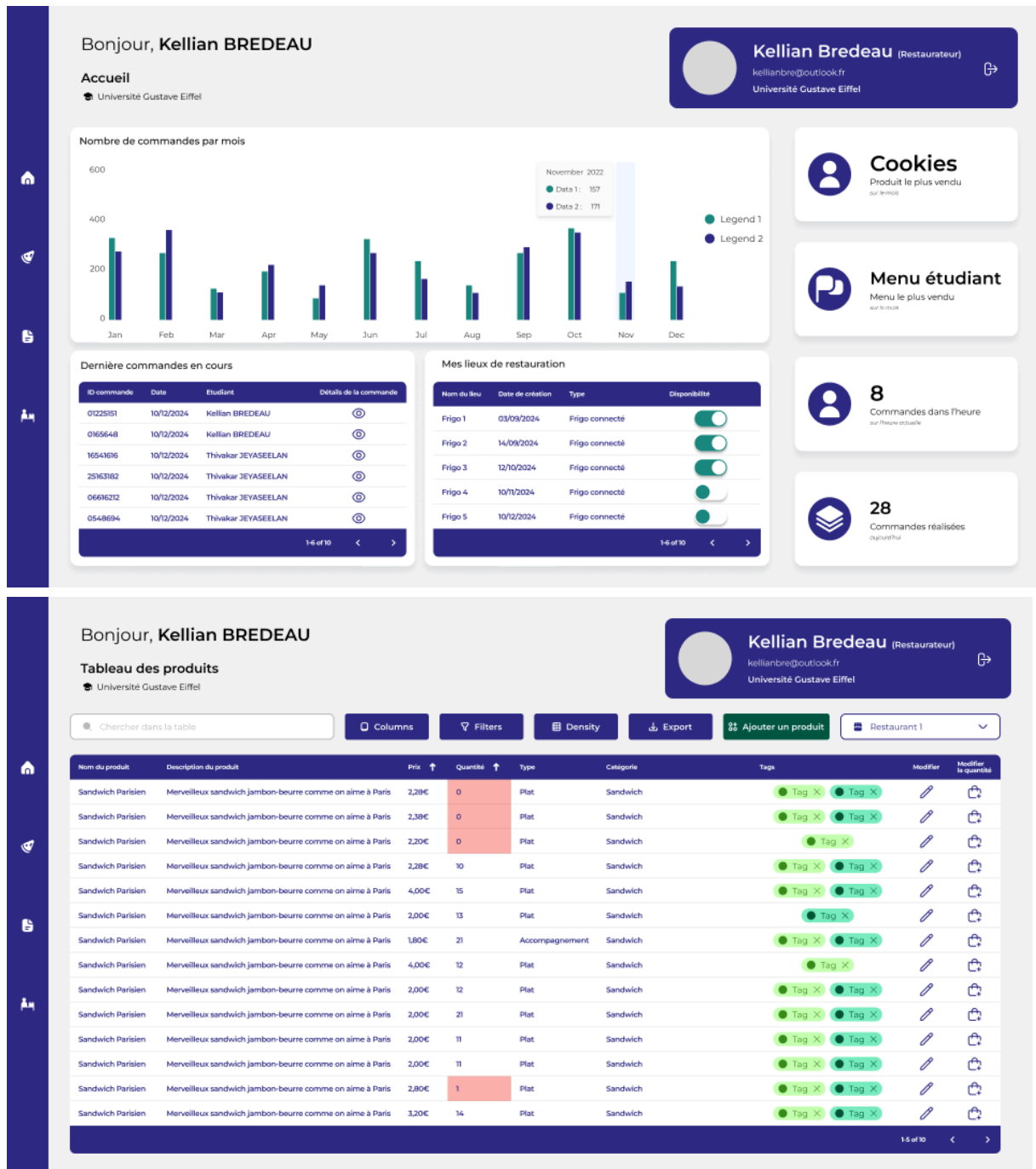
- Tableau de bord (écran 37) : il regroupe des statistiques de l'établissement pour mieux optimiser les espaces.
- Gestion des comptes utilisateurs (écran 38): Cet écran liste les comptes des utilisateurs avec leurs informations de base, facilitant la gestion et le suivi des profils enregistrés.
- Gestion des salles disponibles (écran 39): Cet écran recense les salles disponibles dans l'établissement, facilitant leur consultation et leur modification.

ChillZone - Projet BUT Informatique 3

- Suivi des réservations et signalements (*écran 40*): Cet écran affiche la liste des réservations et des signalements reçus
- Modification du plan de l'établissement (*écran 41*): Cet écran permet de visualiser et de modifier les informations relatives au plan de l'établissement et points d'intérêts.
- Gestion de la Foire aux questions et des liens de l'établissement (*écran 42*): Cet écran permet à l'administrateur de paramétrer sa foire aux questions et ses réseaux sociaux
- Gestion des lieux de restauration (*écran 43*): Cet écran présente un tableau regroupant les différents lieux de restauration affiliés à l'établissement, avec leurs informations associées.

ChillZone - Projet BUT Informatique 3

Ecrans Restaurateur



ChillZone - Projet BUT Informatique 3

Bonjour, **Kellian BREDEAU**

Tableau des menus

Université Gustave Eiffel

Chercher dans la table

Columns

Filters

Density

Export

Ajouter un menu

Restaurant 1

Nom du menu	Description du menu	Prix	Type(s) actifs	Catégories complètes dans le menu	Modifier
Menu McFirst	Menu triple du CROUS	4,00€	Boisson, Entrée, Plat	Tag X Tag X	
Menu McFirst	Menu triple du CROUS	4,00€	Boisson, Entrée, Plat	Tag X	
Menu McFirst	Menu triple du CROUS	4,00€	Boisson, Entrée, Plat	Tag X Tag X	
Menu McFirst	Menu triple du CROUS	4,00€	Boisson, Entrée, Plat	Tag X	
Menu McFirst	Menu triple du CROUS	4,00€	Boisson, Entrée, Plat, Accompagnement, Dessert, Autre	Tag X Tag X	
Menu McFirst	Menu quadruple du CROUS	4,00€	Boisson, Entrée, Plat, Accompagnement, Dessert	Tag X Tag X	
Menu McFirst	Menu triple du CROUS	4,00€	Boisson, Entrée, Plat, Accompagnement, Dessert	Tag X Tag X	
Menu McFirst	Menu quadruple du CROUS	4,00€	Boisson, Entrée, Plat, Accompagnement, Dessert	Tag X	
Menu McFirst	Menu quadruple du CROUS	4,00€	Boisson, Entrée, Plat, Accompagnement, Dessert	Tag X	
Menu McFirst	Menu quadruple du CROUS	4,00€	Boisson, Entrée, Plat, Accompagnement, Dessert	Tag X	
Menu McFirst	Menu triple du CROUS	4,00€	Boisson, Entrée, Plat	Tag X Tag X	
Menu McFirst	Menu triple du CROUS	4,00€	Boisson, Entrée, Plat	Tag X Tag X	
Menu McFirst	Menu triple du CROUS	4,00€	Boisson, Entrée, Plat, Accompagnement, Dessert	Tag X Tag X	
Menu McFirst	Menu triple du CROUS	4,00€	Boisson, Entrée, Plat, Accompagnement, Dessert, Autre	Tag X Tag X	

Bonjour, **Kellian BREDEAU**

Tableau des commandes

Université Gustave Eiffel

Chercher dans la table

Columns

Filters

Density

Export

Restaurant 1

ID	Nom du client	Date	Statut	Total	Début de l'heure de collecte	Voir la commande
0000015	DUPONT	16/02/2024	En attente	6,60 €	12h31	
0000012	CHEVALIER	09/02/2024	En attente	3,30 €	12h31	
0000013	THIERRY	08/02/2024	Prête	3,30 €	12h33	
0000014	SOULEYMANE	08/02/2024	Prête	3,30 €	12h45	
0000015	POGBA	07/02/2024	Prête	3,30 €	12h49	
0000016	DELACROIX	06/02/2024	Complétée	3,30 €	13h00	
0000017	CRISTOPHE	06/02/2024	Complétée	3,30 €	13h00	
0000018	GERARD	18/01/2024	Complétée	3,30 €	13h30	
0000019	JEAN	18/01/2024	Complétée	3,30 €	13h30	
0000010	LEOPOLD	18/01/2024	Complétée	3,30 €	13h45	
0000011	ALAN	18/01/2024	Complétée	3,30 €	13h48	
0000012	FRANCK	18/01/2024	Complétée	3,30 €	14h00	
0000013	DORIAN	18/01/2024	Complétée	3,30 €	14h00	
0000014	BRYAN	18/01/2024	Complétée	3,30 €	14h00	

8

Commandes dans l'heure

sur l'heure actuelle

12

Commandes prévues

d'ici 1h

28

Commandes réalisées

d'ici 1h

Écran d'exemple n°44-47: Pages des fonctionnalités du restaurateur sur PC

Ces écrans regroupent les principales fonctionnalités permettant au restaurateur de gérer son activité au sein de l'établissement :

- Tableau de bord (écran 44): il regroupe des statistiques des restaurants / des frigos connectés pour mieux optimiser les lieux de restauration.
- Gestion des plats (écran 45): Permet d'ajouter, modifier ou supprimer des plats individuels disponibles à la commande.
- Gestion des menus (écran 46): Offre la possibilité de créer et configurer des menus composés de plusieurs types de plats.
- Gestion des commandes (écran 47): Affiche la liste des commandes reçues, avec leur statut (en attente, en cours, préparée, récupérée), afin de faciliter leur suivi et leur traitement.

Ecrans Super Administrateur

Bonjour, **Kellian BREDEAU**

Accueil

Université Gustave Eiffel

Kellian Bredeau (Admin)

kellianbre@outlook.fr

Université Gustave Eiffel

Columns
Filters
Density
Export

Nom Prénom du restaurateur chef	Nom du restaurant	Emplacement du lieu de restauration	Adresse mail du lieu de restauration	Accepter le compte	Refuser le compte
Dupont Olivier	Frigo connecté 101	IUT - Antenne Mame la vallée	dudu@univ-eiffel.fr		
Dupont Olivier	Frigo connecté 201	IUT - Antenne Mame la vallée	dudu@univ-eiffel.fr		
Dupont Olivier	EL RISTORANTE DELA MAMA	IUT - Antenne Mame la vallée	dudu@univ-eiffel.fr		

1-5 of 10 < >

Nom Prénom	Adresse mail	Type	Rôle	Etablissement	Accepter le compte	Supprimer le compte
Dupont Olivier	dudu@univ-eiffel.fr	Administrateur d'établissement	Administrateur d'établissement	IUT - Antenne Mame la vallée		
Dupont Olivier	dudu@univ-eiffel.fr	Restaurateur	Restaurateur	IUT - Antenne Mame la vallée		
Dupont Olivier	dudu@univ-eiffel.fr	Utilisateur	Etudiant	IUT - Antenne Mame la vallée		
Dupont Olivier	dudu@univ-eiffel.fr	Utilisateur	Etudiant	IUT - Antenne Meaux		
Dupont Olivier	dudu@univ-eiffel.fr	Utilisateur	Etudiant	IUT - Antenne Mame la vallée		
Dupont Olivier	dudu@univ-eiffel.fr	Restaurateur	Restaurateur	IUT - Antenne Mame la vallée		
Dupont Olivier	dudu@univ-eiffel.fr	Administrateur d'établissement	Administrateur d'établissement	IUT - Antenne Mame la vallée		
Dupont Olivier	dudu@univ-eiffel.fr	Utilisateur	Professeur	IUT - Antenne Mame la vallée		
Dupont Olivier	dudu@univ-eiffel.fr	Utilisateur	Professeur	IUT - Antenne Meaux		

1-5 of 10 < >

Ecran d'exemple n°48(-49): Page supplémentaire du super-administrateur sur PC

Cet écran est nécessaire au super-administrateur pour plus facilement accepter ou refuser les demandes de restaurants. Il peut également visualiser tous les comptes back-office pour consultation. Lors du développement il a été divisé en deux écrans pour chaque fonctionnalité décrite.

Pour consulter le prototypage de la maquette, veuillez vous référer à [l'annexe 5 - Maquette du projet](#).

3.2 Architecture du système

Organisation modulaire

Chaque projet est structuré en dossiers séparés : web, mobile et API. Cette dissociation garantit une indépendance manifeste entre les composantes du système, facilite la gestion des fichiers et autorise les équipes à se focaliser sur leur secteur précis sans crainte d'interférence.

Chaque structure de dossier a été élaborée pour distinguer clairement les responsabilités et simplifier la complexité. Le backend, conçu avec Django, est organisé en applications séparées, chaque application étant consacrée à une fonction précise comme la gestion des utilisateurs, des commandes ou des réservations. Cette méthode assure une maintenabilité supérieure et une capacité d'évolution améliorée.

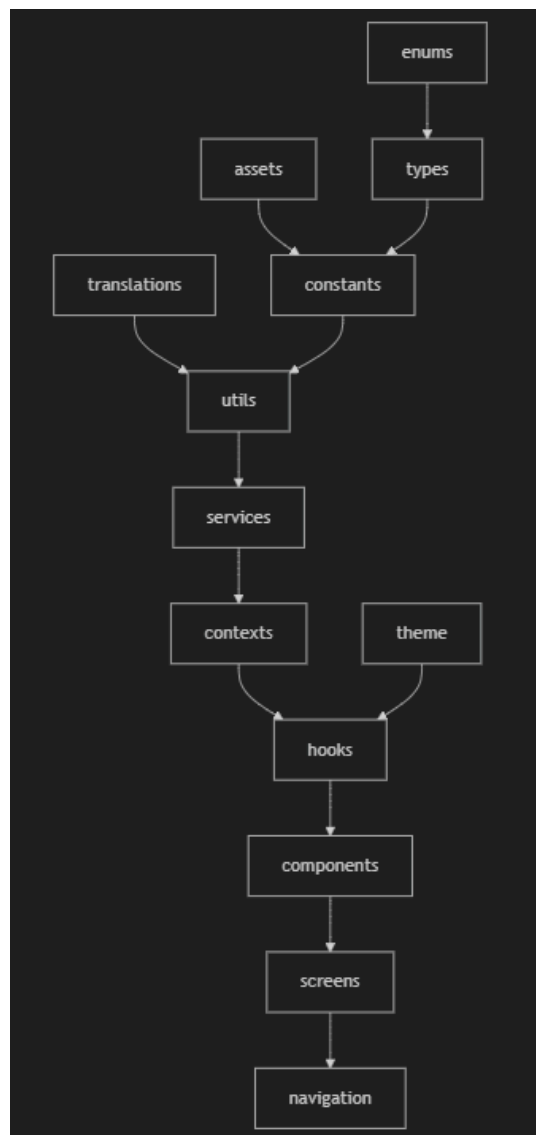
Du côté du web et de l'application mobile, réalisées respectivement avec React et React Native, le code est organisé en composants réutilisables et bien isolés. Cette division facilite la lisibilité du code, sa maintenance et sa réutilisation dans différents contextes. Par ailleurs, les interfaces API sont centralisées pour assurer une communication simple, cohérente et efficace entre le backend et les frontends.

Grâce à cette organisation, les bases du projet sont solides, permettant un développement futur fluide et collaboratif. Chaque membre de l'équipe peut travailler sereinement sur une partie spécifique du système sans risquer de générer des conflits avec les autres.

Architecture des dossiers

L'architecture des dossiers a été organisée pour garantir une structure claire, maintenable et facile à comprendre pour l'ensemble des développeurs. Voici une image représentative de l'architecture des dossiers.

Illustration 2 : Architecture des dossiers



- **enums** : Contient des énumérations utilisées pour définir des constantes spécifiques avec des valeurs prédéfinies.
- **types** : Regroupe toutes les définitions de types TypeScript (ou équivalents) permettant de structurer et typer correctement les données manipulées dans l'application.
- **assets** : Stocke les ressources statiques telles que les images, icônes, polices, et autres fichiers multimédias nécessaires au projet.

- **constants** : Contient toutes les constantes globales utilisées dans le projet, comme des configurations ou des valeurs récurrentes.
- **translations** : Gère tous le wording de l'application et peut gérer des fichiers de traduction pour supporter plusieurs langues.
- **utils** : Réunit les fonctions utilitaires, c'est-à-dire des outils génériques réutilisables dans différentes parties du projet.
- **services** : Contient la logique pour interagir avec des API externes ou internes, ainsi que la gestion des appels réseau et des services applicatifs.
- **contexts** : Regroupe les contextes (React Context) utilisés pour partager des états globaux à travers l'application de manière performante.
- **theme** : Définit le style global de l'application, incluant les couleurs, polices et autres variables visuelles partagées.
- **hooks** : Contient des hooks personnalisés, permettant de simplifier et de réutiliser la logique métier dans le projet.
- **components** : Regroupe les composants réutilisables de l'application, tels que des boutons, des champs de formulaire ou des cartes.
- **screens** : Contient les différentes pages de l'application, chaque écran représentant une fonctionnalité ou vue spécifique.
- **navigation** : Gère toute la logique liée à la navigation de l'application, que ce soit au niveau des onglets, des piles ou des routes.

Cette hiérarchie assure une organisation claire avec une séparation logique entre les différentes responsabilités, ce qui facilite le développement, la maintenance et l'évolutivité du projet.

3.3 Conception de la base de données

Ce projet dispose d'une base de données spécifiquement élaborée pour soutenir un système de commande et collecte, la réservation d'espaces, la gestion des profils utilisateurs, la navigation par plan, ainsi que l'intégration d'un calendrier. Le modèle conceptuel de données (MCD) illustre les liaisons logiques entre les entités clés, comme les établissements de restauration, les clients, les mets et les commandes. De son côté, le modèle logique de données (MLD) adapte ces liaisons en une structure relationnelle appropriée pour la mise en œuvre avec Django et MySQL.

La table `compte_utilisateur` est utilisée pour la gestion des utilisateurs et contient des informations essentielles comme le prénom, le nom de famille, l'adresse email, le rôle (client, administrateur ou gérant), ainsi que des données d'accès sécurisées. Cette table est liée aux établissements pour simplifier la gestion hiérarchique des lieux par les administrateurs ou gérants.

Dans le système de click and collect, les entités menu et plat occupent une position centrale au sein de l'application. La table des produits contient les détails de chaque article proposé (nom, description, prix, stock, etc.), alors que le menu rassemble ces articles en collections liées à des emplacements de restauration précis (`lieu_restaurations`). Les tables `commande` et `ligne_commande` sont utilisées pour gérer les commandes. La première conserve les informations principales de l'achat (mode de paiement, total, QR code pour la récupération), tandis que la seconde précise les plats commandés ainsi que leurs quantités.

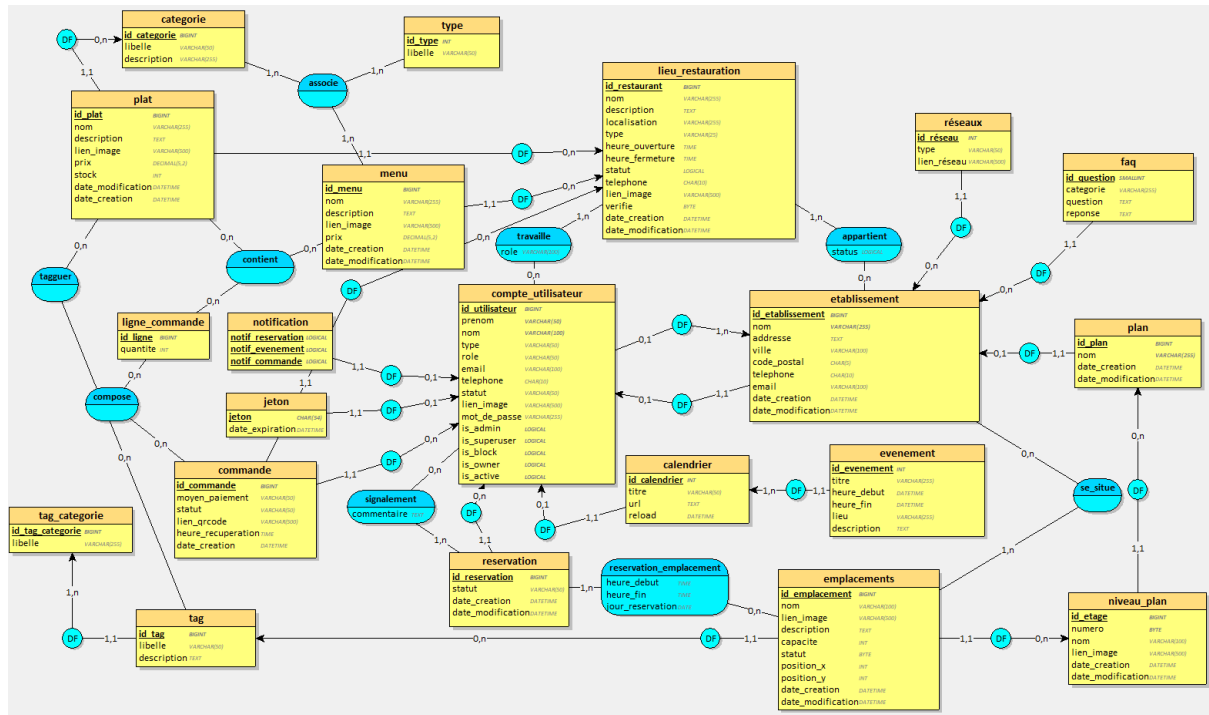
La table des réservations permet de gérer les réservations de salles, comprenant le statut, les dates et les horaires. La table `emplacements` fournit des détails sur les lieux disponibles, y compris leur capacité, leur localisation et leur statut (libre ou occupé). Cette option est particulièrement pratique pour la réservation de salles ou d'espaces destinés à des événements.

Le système de navigation par plan s'appuie sur les tables « plan » et « niveau_plan », qui servent à illustrer les plans physiques des structures. Chaque plan est associé à un établissement donné, tandis que les diverses strates d'un plan (tels que les étages) sont précisés dans `niveau_plan`. Cette structure inclut la table des emplacements afin de localiser exactement les espaces réservables sur le plan.

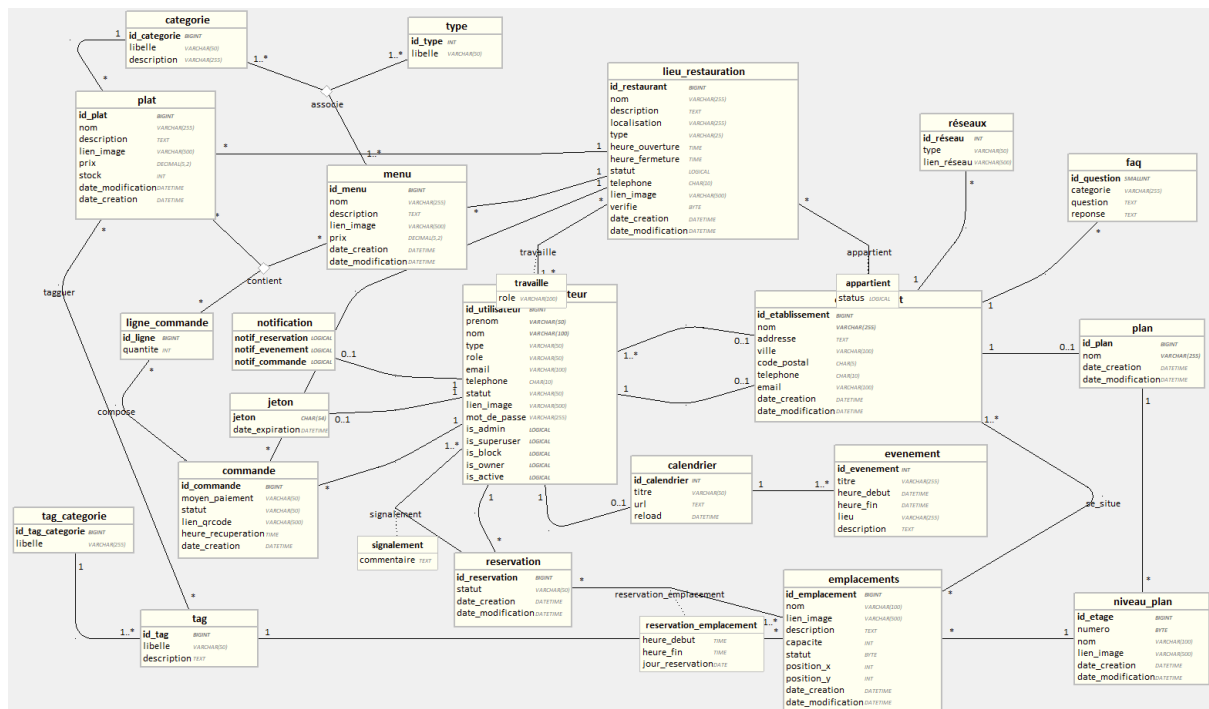
L'organisation des événements se réalise via la table `calendrier`, qui centralise les événements liés à un utilisateur ou à une entité. La table des événements fournit une perspective précise des activités planifiées, incluant des informations sur les temps, les lieux et les descriptions des événements.

Techniquement, l'API est élaborée en utilisant Django et Django Rest Framework (DRF), ce qui facilite une interaction harmonieuse avec les interfaces frontales. Les endpoints REST sont structurés selon leurs fonctionnalités. Chaque endpoint bénéficie d'un système de permissions sur mesure, avec des rôles spécifiques attribués pour limiter l'accès (client, administrateur, manager). Les serializers de DRF simplifient la transformation des objets de base de données en JSON, garantissant ainsi leur adéquation avec les clients Front-End tels que React.

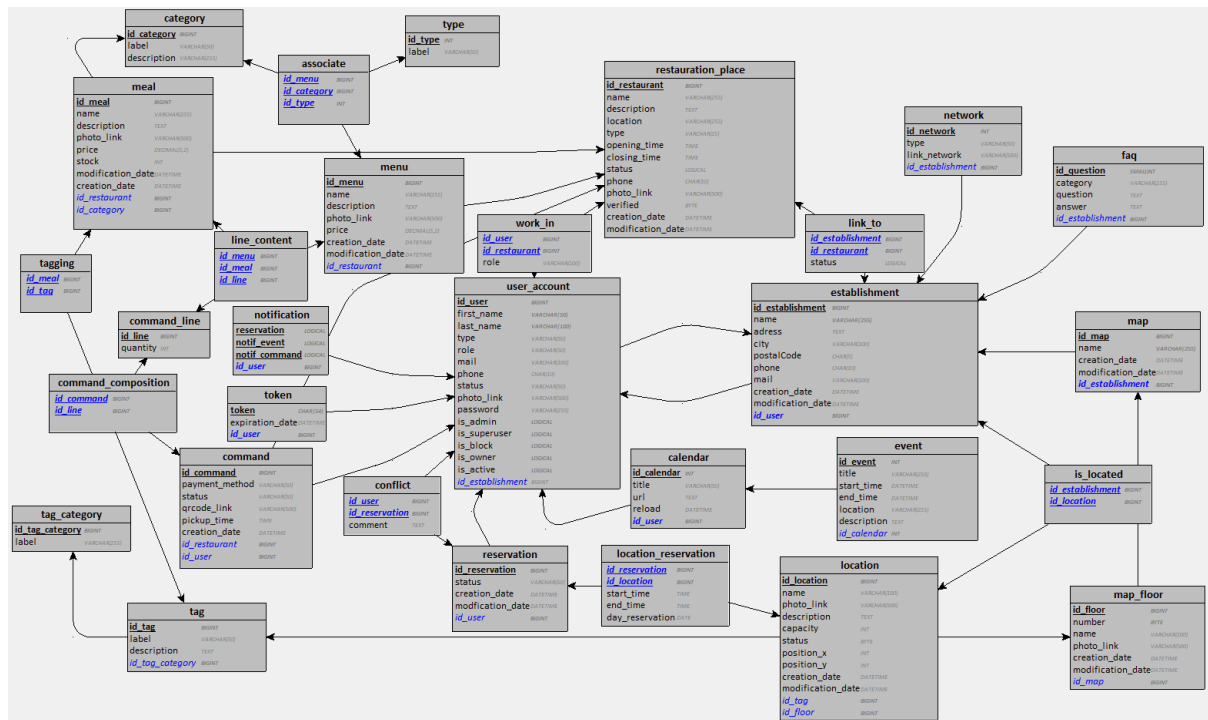
Modèle Conceptuel de Données (MCD)



Langage de Modélisation Unifié (UML)



Modèle Logique de Données (MLD)



Modèle Entités / Associations (MEA)

Entités principales

Restauration_place (id_restaurant (PK), name, description, location, type, opening_time, closing_time, status, phone, photo_link, verified, creation_date, modification_date)

Menu (id_menu (PK), name, description, photo_link, price, creation_date, modification_date, id_restaurant (FK))

Category (id_category (PK), label, description)

Command_line (id_line (PK), quantity)

Tag_category (id_tag_category (PK), label)

Type (id_type (PK), label)

Meal (id_meal (PK), name, description, photo_link, price, stock, modification_date, creation_date, id_restaurant (FK), id_category (FK))

Tag (id_tag (PK), label, description, id_tag_category (FK))

Establishment (id_establishment (PK), name, address, city, postalCode, phone, mail, creation_date, modification_date)

Map_floor (id_floor (PK), number, name, photo_link, creation_date, modification_date)

ChillZone - Projet BUT Informatique 3

Map (id_map (PK), name, creation_date, modification_date, id_establishment (FK))

User_account (id_user (PK), first_name, last_name, type, role, mail, phone, status, photo_link, password, is_admin, is_superuser, is_block, is_owner, is_active)

Reservation (id_reservation (PK), status, creation_date, modification_date, id_user (FK))

Location (id_location (PK), name, photo_link, description, capacity, status, position_x, position_y, creation_date, modification_date, id_tag (FK), id_floor (FK))

Command (id_command (PK), payment_method, status, qrcode_link, pickup_time, creation_date, id_restaurant (FK), id_user (FK))

FAQ (id_question (PK), category, question, answer, id_establishment (FK))

Calendar (id_calendar (PK), title, url, reload, id_user (FK))

Event (id_event (PK), title, start_time, end_time, location, description, id_calendar (FK))

Notification (id_notification (PK), reservation, notif_event, notif_command, id_user (FK))

Network (id_network (PK), type, link_network, id_establishment (FK))

Token (token (PK), expiration_date, id_user (FK))

Associations

Link_to (id_establishment (FK), id_restaurant (FK), status)

Work_in (id_user (FK), id_restaurant (FK), role)

Associate (id_menu (FK), id_category (FK), id_type (FK))

Line_content (id_menu (FK), id_meal (FK), id_line (FK))

Is_located (id_establishment (FK), id_location (FK))

Location_reservation (id_reservation (FK), id_location (FK), start_time, end_time, day_reservation)

Tagging (id_meal (FK), id_tag (FK))

Command_composition (id_command (FK), id_line (FK))

Conflict (id_user (FK), id_reservation (FK), comment)

4. Développement

Nous nous sommes penchés sur la mise en place de l'architecture Front-End et l'installation des frameworks en même temps que la préparation de la maquette afin de commencer rapidement le développement de la méthode Agile.

4.0. Méthodologie de projet appliquée

La méthodologie adoptée pour la gestion de ce projet s'est articulée autour d'une approche Agile, favorisant la flexibilité, la communication constante et l'adaptabilité aux changements. Cette méthodologie a été mise en pratique en divisant le projet en itérations courtes et fonctionnelles. Chaque itération a été planifiée sous forme de "sprints" avec des objectifs spécifiques à atteindre.

Pour organiser notre travail, nous avons utilisé un système de gestion de projet basé sur des tickets. Chaque "Epic Story" identifiée dans le projet a été décomposée en une série de tickets plus petits et spécifiques. Ces tickets ont été attribués à des membres de l'équipe en fonction de leurs compétences et de leurs domaines d'expertise respectifs. Chaque ticket comprenait des détails sur la tâche à accomplir, les dépendances éventuelles et une estimation du temps nécessaire à sa réalisation. Les tickets reprennent soit les fonctionnalités à implémenter soit les users stories du backlog.

Le suivi des tickets s'est fait via Jira, une plateforme de gestion de projet où chaque membre pouvait mettre à jour l'état d'avancement de ses tâches. La priorisation des tâches s'est basée sur la valeur métier et la complexité technique estimée lors des sessions de Poker Planning.

Pour chaque sprint réalisé dans le cadre de la méthode Agile, notre équipe a minutieusement élaboré des sessions de Sprint Planning et de rétrospectives de Sprint. Ces réunions essentielles sont cruciales pour le bon déroulement du processus de développement et contribuent à l'amélioration continue de notre équipe. En termes de comitologie, nous avons établi une réunion d'équipe par sprint ainsi que plusieurs réunions avec la MOA ou notre professeur référent pour affiner le périmètre du projet.

Cette approche par tickets a permis une répartition claire des responsabilités, une meilleure gestion du temps et une collaboration efficace au sein de l'équipe, contribuant ainsi à une progression méthodique du projet et à la réalisation des objectifs fixés pour chaque itération.

Nous avons également utilisé Confluence comme espace centralisé de documentation pour assurer un suivi structuré du projet. Chaque sprint fait l'objet d'une réunion "stand-up" documentée sur Confluence, où le responsable de la documentation répertorie les points positifs, les axes d'amélioration et les actions à mettre en place pour les itérations suivantes. Grâce à cette documentation, nous avons pu capitaliser sur les apprentissages de chaque sprint et renforcer l'efficacité de notre méthodologie Agile. Toutes notes de réunions et notes de d'équipe sont contenues dans confluence pour les relier facilement à Jira.

ChillZone - Projet BUT Informatique 3

Pour assurer une organisation efficace et une répartition équilibrée des responsabilités, nous avons défini six domaines clés pour le projet, chacun étant attribué à un responsable principal et un backup pour garantir la continuité en cas d'indisponibilité.

Domaine	Responsable	Backup
Gestion de projet	Helena	Taha
Design	Thivakar	Kellian
Front-End	Elias	Thivakar
Back-End	Kellian	Loic
Tests	Taha	Elias
Documentation	Loic	Helena

Tableau 1 : Répartition des responsabilités dans l'équipe

Cette structuration (*tableau 1*) a permis une meilleure gestion des tâches et une collaboration fluide entre les membres de l'équipe, tout en assurant une flexibilité dans la prise en charge des différentes missions du projet (*illustration 3*).

Répartition du travail par domaines de tickets

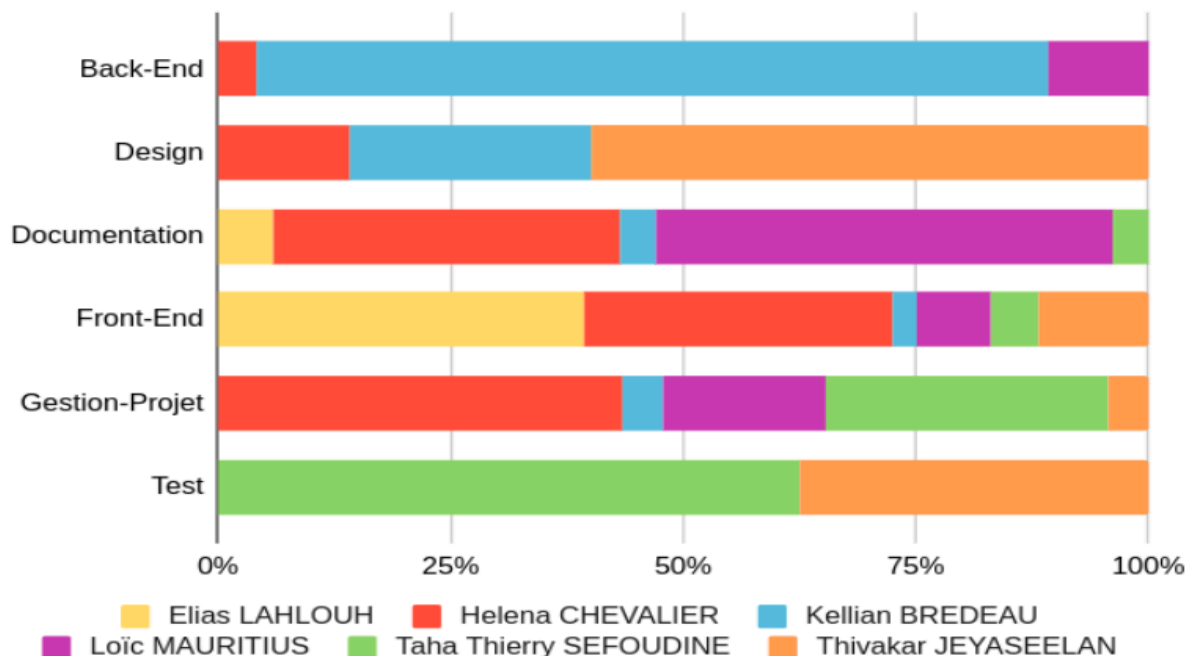


Illustration 3 : Graphique montrant la répartition globale des tickets par types de tickets (étiquettes)

Pour notre méthodologie des tests il est recommandé de se référer à la partie 5 de ce rapport [5. Tests et validation](#).

4.1. Étapes de développement

Tout d'abord, nous nous sommes concentrés sur la mise en place de l'architecture du projet. Nous avons créé différents dossiers pour séparer le Back-end, le Front-End web et le Front-End mobile.

En Front-End, on a commencé à installer React Native dans le dossier mobile et React TypeScript dans le dossier web.

Pour respecter l'architecture en Atomic Design, on a créé les différents dossiers pour chaque responsabilité de la hiérarchie de l'architecture système.

Nous avons par ailleurs implémenter des données statiques pour rendre notre site multi-language ou simuler des retours avec le Back-End.

Cette architecture nous a permis de nous lancer directement dans le développement au niveau du Front-End.

Pour la partie Back-End, nous avons créé les objets permettant de déployer rapidement la base de données. De plus, nous avons configuré l'API Django pour gérer les accès et les droits.

Pendant que l'on se concentrait sur la maquette, le responsable Back-End et le responsable Front-End ont été chargés de créer cette architecture dans le repository actuel afin de gérer les différentes versions du projet et de collaborer efficacement au sein de l'équipe.

Ensuite, nous nous sommes réparti les tâches de sorte à mettre deux personnes sur le Front-End mobile et deux personnes sur le Front-End web et enfin, deux personnes sur le Back-End. Le but étant de se répartir les tâches équitablement dans le projet, certaines personnes peuvent transmuter lorsque leurs tâches sont finis pour aider les autres au besoin.

Après une répartition claire des tâches, nous avons entamé l'implémentation des fonctionnalités attendues en dissociant le développement du Front-End et du Back-End. Cette approche permet à chacun de se concentrer sur sa mission spécifique, tout en garantissant une meilleure modularité et évolutivité du projet. Cela facilite grandement l'ajout ou la modification de composants sans impacter l'ensemble de l'application. De plus, elle simplifie les tests et favorise une collaboration efficace entre les équipes, en limitant les interférences liées aux problématiques propres à chaque domaine.

Pour cela, la partie Back-End, codée en python, utilise Django avec la bibliothèque Rest Framework permettant une utilisation plus optimisée des fonctions et modules lors de la mise en place une API RestFul.

Niveau Front-End, des fichiers .JSON ont été créés simulant le comportement de renvoi de l'API. Nous pouvons donc gérer l'affichage des informations sans grande difficultés et se concentrer sur des aspects tels que le design, l'accessibilité ou encore le développement en Atomic Design.

Côté Web, nous avons mis en place l'ensemble des interfaces pour les fonctionnalités clés, telles que l'authentification, les tableau de bord complet des administrateurs, du super administrateur et des restaurateurs ainsi que les modals, en nous concentrant sur une interface ergonomique, une navigation intuitive et une compatibilité multi-plateforme, afin de garantir une expérience utilisateur fluide et cohérente.

ChillZone - Projet BUT Informatique 3

Le développement Front-End côté mobile s'est également déroulé sans grande difficultés, nous avons implémenté tous les écrans d'authentification, de Click and Collect, de réservation de bax acoustique ou encore de calendrier avec tous les écrans autour afin d'assurer une correspondance optimale avec l'application finale.

Au deuxième semestre, nous nous sommes concentrés sur l'intégration du Back-end dans la partie Front-End. Cette partie a pris du temps car elle repose essentiellement sur du débogage (Ajuster le format des données récupérer et les potentiels problèmes mineurs que nous pouvions rencontrer comme des bugs d'affichage ou de type).

Nous gérons, à la connexion, les données utilisateur sous forme de cookies avec le localStorage (côté web) ou Async Storage (côté mobile). Nous prenons alors les données utilisateurs renvoyées par l'API en cas de connexion réussi mais également le token d'accès pour communiquer avec l'API.

Titre du sprint	Dates du sprint	Fonctionnalités / Tâches majeures
Lancement du projet	2024/10/08 2024/10/20 (12 jours)	- Répartition des rôles - Contact avec la MOA - Réflexion sur les choix techniques
Refonte du design & documents de besoins	2024/10/21 2024/11/11 (21 jours)	- Réécriture du cahier des charges fonct. - Refonte intégrale de la maquette Figma - Recueil des besoins utilisateurs
Design et Architecture Technique	2024/11/12 2024/11/31 (18 jours)	- Finalisation et prototypage de la maquette - Conception de la base de données - Mise en place de l'architecture technique
Lancement du développement, authentification	2024/12/01 2024/12/16 (15 jours)	- Création de l'API Django REST - Répartition équipe web et équipe mobile - Création des fonctionnalités de connexion
Réservation de repas et Gestion de l'administrateur	2024/12/16 2024/12/26 (10 jours)	- Développement Interface web administrateur d'établissement - Développement de la réservation de repas en click and collecte - Développement de l'API en ORM
Réservation de salles et espace utilisateur	2024/12/27 2025/01/05 (10 jours)	- Développement de la réservation de salles - Approfondissement de l'API - Développement du compte utilisateur
Consultation du planning et Gestion du restaurateur	2025/01/06 2025/01/11 (6 jours)	- Développement de l'interface restaurateur - Développement de l'espace calendrier - Développement des notifications

Intégration de l'API à l'application web	2025/01/12 2025/01/31 (19 jours)	- Intégration de l'API avec l'application web - Création de l'environnement de test (Docker) - Corrections et améliorations UX/UI
Intégration de l'API à l'application mobile	2025/02/01 2025/02/15 (15 jours)	- Connexion de l'application mobile à l'API - Développement des fonctionnalités mobiles - Finalisation de l'API
Recherche dans le plan interactif	2025/02/16 2025/02/28 (14 jours)	- Développement de la recherche interactive - Amélioration de l'ergonomie du plan - Intégration des données dynamiques
Transfert de connaissances	2025/03/01 2025/03/08 (8/ jours)	- Documentation technique et fonctionnelle - Tests utilisateurs & corrections mineures - Finalisation et passation du projet

Tableau 1 : Différents sprint du semestre 1 avec les fonctionnalités majeures

4.2. Défis rencontrés

Au niveau organisationnel, nous avons eu des difficultés à finaliser certains sprints dans les délais prévus car il y a grand nombre de tâches à réaliser dans des délais très courts.

Au niveau technique, nous apprenons constamment les frameworks React Native et Django à travers leurs utilisations. Ce point était un peu bloquant au début du projet mais ne nous pose plus de problème au fur et à mesure du développement.

Le seul souci majeur que nous avons rencontré durant ce projet est la portée du projet qui est vraiment trop importante par rapport à un développement de quelques mois. Il nécessite beaucoup de temps et même pour un POC, il est extrêmement complet avec ses 8 fonctionnalités principales.

4.3. Solutions apportées

Nous adaptons nos priorités et maintenons une gestion efficace du projet en réorganisant les tâches et en nous concentrant sur l'essentiel pour respecter les objectifs globaux. Cette approche nous a permis de progresser tout en conservant une bonne qualité de travail.

Nous nous référons d'ailleurs beaucoup aux documentations des frameworks si besoin et savons nous adapter.

Il nous tient à cœur de faire notre maximum durant ce projet malgré sa complexité.

4.4. Code source et démonstration finale

Pour la partie Front-End, l'architecture du projet sous forme de dossier se présente ainsi pour le web et le mobile :

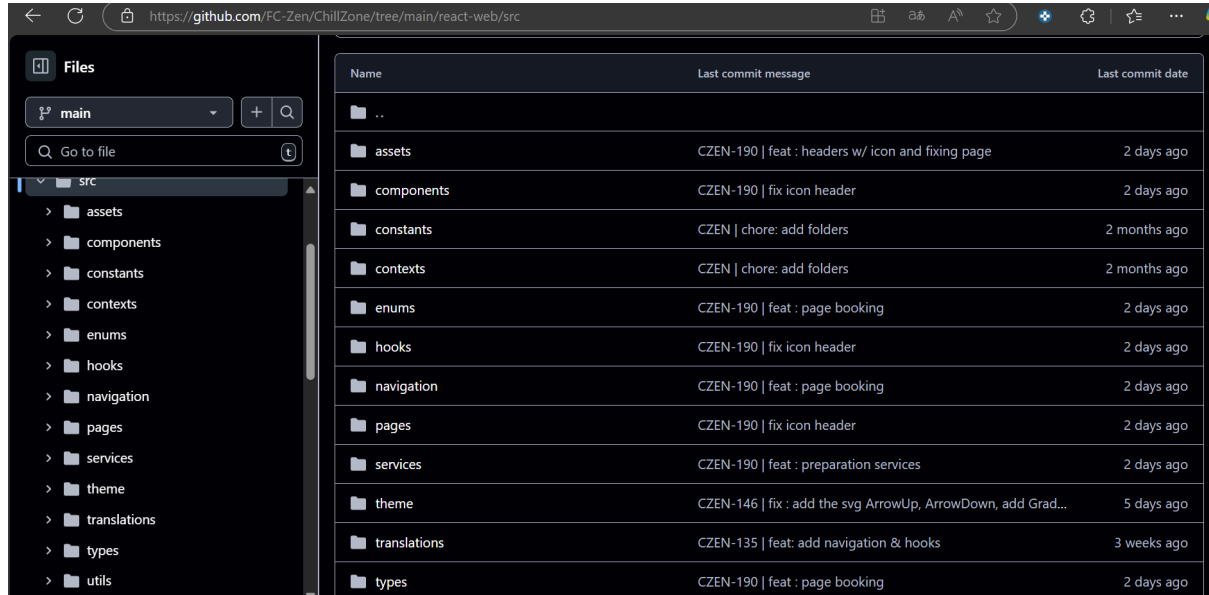


Illustration 4 : Capture d'écran représentant l'architecture des dossiers en Front-end (web et mobile)

Avec cette architecture et de l'Atomic design, nous arrivons à créer des composants réutilisables et les assembler pour ensuite créer des pages élaborées et complexes. Voici un exemple de développement en Atomic design, qui est une méthode de conception qui décompose les interfaces en éléments fondamentaux (atomes, molécules, organismes) pour créer des systèmes modulaires et cohérents.

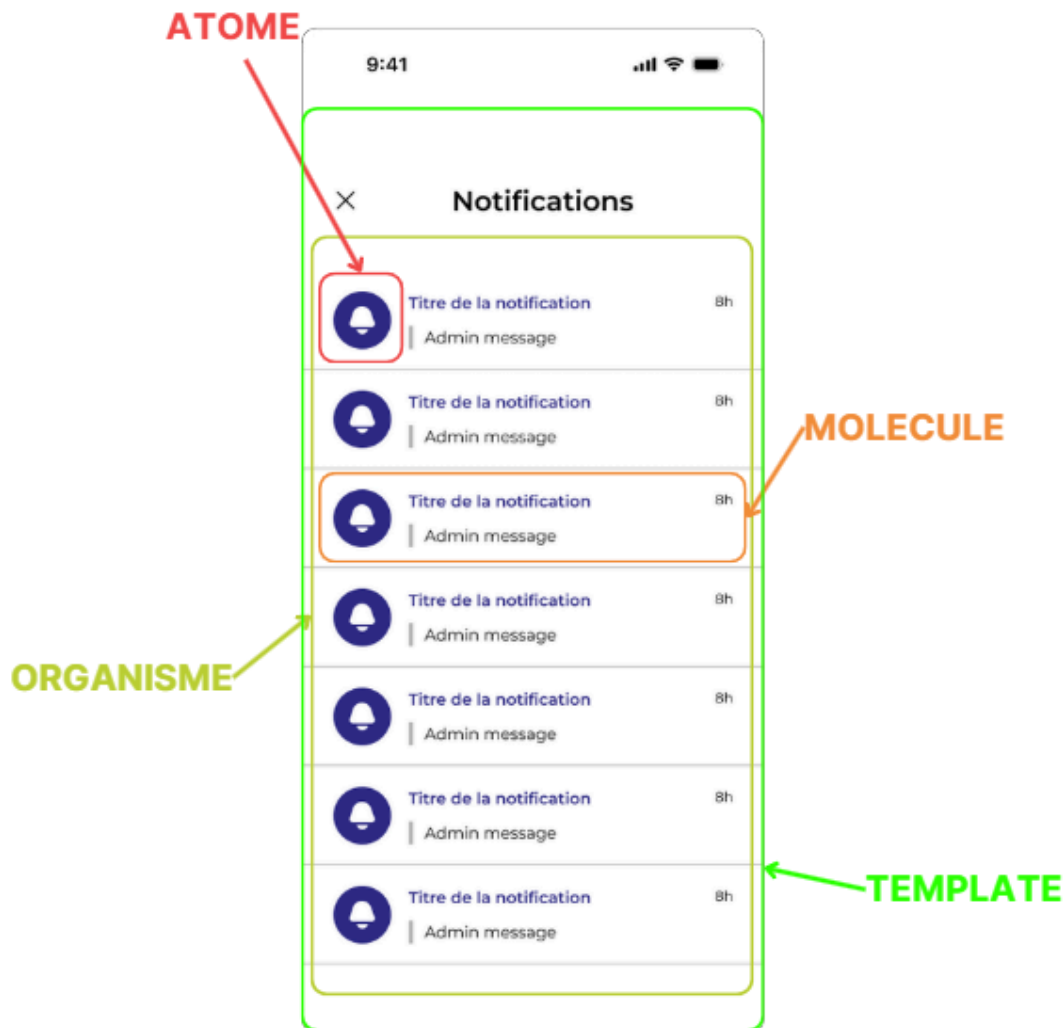


Illustration 5 : Capture d'écran représentant la structure de l'Atomic Design

L'API est structurée en plusieurs composants essentiels, chacun ayant un rôle spécifique. Les modèles représentent les entités de l'API sous forme de classes définissant les structures de données et leur relation avec la base de données. Les serializers permettent de convertir ces données entre différents formats, comme JSON, afin de faciliter les échanges entre le back-end et le Front-End. Les fichiers de migration assurent la gestion des modifications de la base de données, garantissant une synchronisation cohérente entre les changements apportés au modèle et la base elle-même.

Les vues gèrent les requêtes et les réponses en utilisant des classes ou des fonctions qui définissent la logique de traitement des données reçues et envoyées. La couche de services centralise la logique métier réutilisable, notamment pour des fonctionnalités comme l'envoi d'e-mails ou d'autres interactions complexes. Enfin, les fichiers racine sont nécessaires pour configurer l'API, incluant les paramètres de connexion à la base de données, le routage des requêtes ou encore la gestion des dossiers du projet.

5. Tests et validation

Pour vérifier que le code est propre et fonctionnel, nous appliquons la méthode de VVT ; Vérification, Validation & Test des logiciels. Puisque nous sommes dans un projet utilisant la méthode Agile, les concepts de Validation, Vérification et Test sont intégrés tout au long du cycle de développement.

La vérification consiste à s'assurer que le logiciel est construit correctement en respectant les exigences et les bonnes pratiques de développement. Dans notre équipe, la revue de code se fait principalement via des GitHub Pull Requests, ce qui permet une validation du code avant son intégration dans la branche principale.

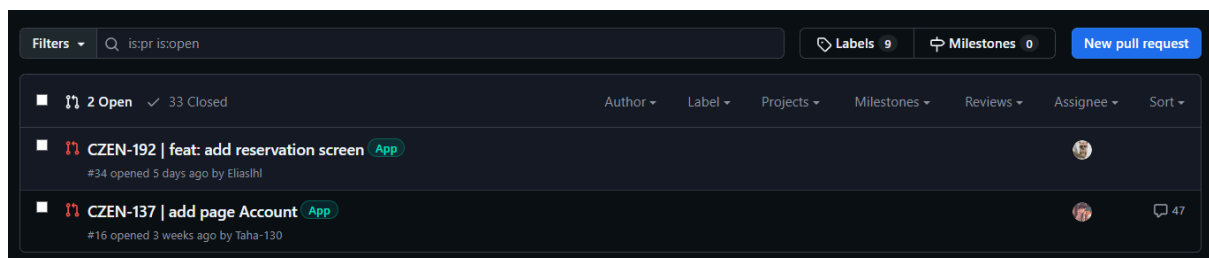


Illustration 6 : Capture d'écran montrant les pull-requests actuellement ouverte, avec le nom du ticket Jira relié, le nom de la page, la partie du projet concerné, la personne assignée et le nombre de commentaires fait en vérification du code.

Chaque développeur soumet son code sous forme de Pull Request (PR) (Illustration 6) en indiquant la partie du projet concernée (App Mobile, Web ou API). Le responsable de cette section est alors chargé d'examiner le code (faire une "review") afin de garantir son respect des règles de codage définies par l'équipe, ainsi que sa conformité fonctionnelle et esthétique par rapport aux spécifications et aux maquettes conçues avec la MOA. Si la review est validée, le développeur peut alors envoyer ses modifications sur la branche principale du projet. Si la review nécessite des changements, il devrait corriger et demander une nouvelle vérification. Tant que la review n'est pas validée elle ne peut pas être implémentée dans la branche principale (Illustration 7).

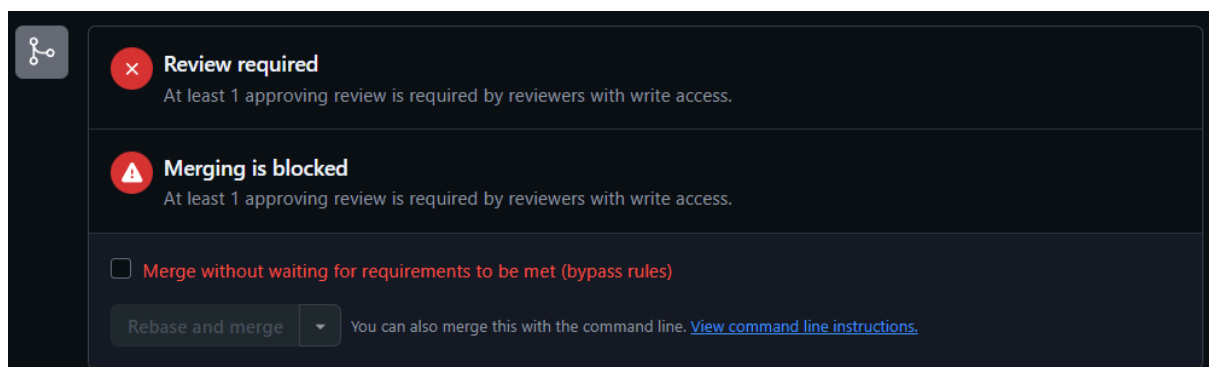


Illustration 7 : Capture d'écran montrant l'obligation d'avoir un réviseur sur chaque branche sans quoi il est impossible de "merge" la branche sur le main.

Ce processus remplace en partie l'intégration continue (CI/CD), car nous ne disposons pas d'un pipeline automatisé de tests et de déploiement.

Pour la validation, la cheffe de projet ou les membres de l'équipe révisent régulièrement les pages finies dans le sprint pour s'assurer que le code respecte toutes les fonctionnalités demandées du cahier des charges. Durant la validation, l'équipe va aussi se concentrer sur les tests exploratoires, qui sont axés sur les améliorations non prévues dans le cahier des charges, comme des changements qui visent à améliorer l'expérience utilisateur.

La connexion à l'API permet de contrôler l'intégration des données dynamiques mais aussi des autorisations associées aux fonctionnalités.

Les tests sont effectués à la fois en local et dans un environnement simulé de production, déployé temporairement sur un serveur hébergé dans une image Docker durant le deuxième semestre. Cette approche permet de réaliser des tests d'intégration approfondis et d'anticiper d'éventuelles anomalies liées au déploiement sur serveur après reprise du POC.

Dans le cadre des tests utilisateurs et de la validation de l'interface, nous avons recours à Expo GO, une solution permettant de relier l'application mobile et de pouvoir la tester sur un téléphone. Cette approche offre la possibilité de tester l'application dans des conditions d'utilisation comme la réalité, en tenant compte des interactions tactiles et des performances sur différents appareils. En effet, réaliser les tests utilisateurs sur mobile permet d'identifier d'éventuels dysfonctionnements ergonomiques ou techniques, pour ensuite ajuster les éléments de l'interface et de garantir une expérience utilisateur agréable.

Ces méthodologies nous confèrent la possibilité de tester l'expérience utilisateur dans un environnement réel, sans qu'il soit nécessaire d'effectuer un déploiement préalable sur des appareils physiques.

Pour les tests logiciels, nous mettrons en œuvre l'approche Test-Driven Development (TDD) (illustration 8) en utilisant des bibliothèques spécialisées comme Jest pour les tests en React.

Les tests unitaires permettent de valider le bon fonctionnement de chaque composant isolément, tandis que les tests d'intégration assurent la cohérence des différentes parties du système. De plus, nous avons utilisé React Native Testing Library pour simuler les interactions utilisateurs et garantir un comportement réaliste des composants.

L'utilisation de mocks pour simuler les dépendances, la vérification systématique des résultats attendus nous ont permis d'optimiser la qualité du code et de prévenir d'éventuelles régressions au fil du développement. Pour plus d'informations sur cette partie, il est conseillé de lire le [8 - Documentation des tests utilisateurs](#) dans la partie Annexe.

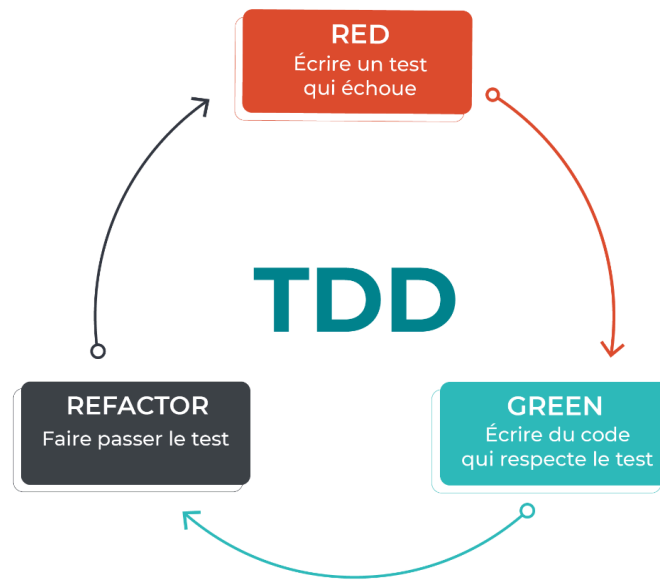


Illustration 8 : schéma explicatif de la méthode Test-Driven Development (TDD)

L'ensemble des tests mis en place dans notre projet nous permet d'assurer la qualité, la stabilité et la conformité du développement tout au long du cycle de vie du logiciel. En appliquant la méthode VVT (Vérification, Validation et Test) et en intégrant les revues de code via GitHub, nous garantissons un code propre et conforme aux exigences définies. Les tests effectués sont fiables grâce aux simulations d'environnement de production réalisés et ont permis une amélioration significative du projet par le développement de notre esprit critique.

Conclusion et perspectives

Le projet ChillZone s'est déroulé sur une année académique complète, marquée par une évolution constante d'octobre à mars. La première phase du projet a nécessité une refonte des fondements du projet pour répondre aux nouvelles exigences établies par la maîtrise d'ouvrage (MOA). Cette période a été principalement consacrée à l'élaboration d'une interface utilisateur intuitive et fonctionnelle. Si au premier semestre nous avons réalisé l'ensemble des écrans sans données, le second semestre a été consacré à l'intégration de données dynamiques dans le POC. Pour le rendu final, les fonctionnalités essentielles de l'application ont été développées et intégrées avec succès, transformant les prototypes initiaux en une solution cohérente et fonctionnelle.

Le bilan du projet met en évidence une progression significative malgré les défis rencontrés et l'ampleur du projet. En effet, le projet comprend de nombreuses fonctionnalités diverses, nécessitant 42 tables implémentées en ORM dans la base de données, 69 user stories répartis dans 8 Epics, et pas moins de 207 tickets JIRA créés pour assurer une gestion des tâches et des évolutions. Le suivi des tickets Jira, associé à des sprints bien définis, contribue à la bonne organisation du projet.

L'évolution du projet a été marquée par une amélioration importante des documents de spécifications (cahier des charges et maquettes). Toute cette organisation a contribué à renforcer la qualité et la pertinence des fonctionnalités mises en place pour le POC.

Les choix technologiques, notamment l'utilisation de React et de React Native pour le Front-End et Django pour le back-end, ont su être adoptés par l'équipe en seulement un mois de développement. Certains ajustements restent nécessaires pour optimiser certaines fonctionnalités et garantir leur bon fonctionnement lors de l'intégration.

L'adoption d'une approche TDD (Test-Driven Development) s'est révélée être un choix judicieux, permettant d'assurer la qualité et la fiabilité du code produit. Cette méthodologie a facilité la détection précoce des bugs et optimisé le processus de développement.

Notre projet étant à l'heure actuelle un POC (Proof of Concept), visant à démontrer l'intérêt potentiel que pourrait représenter le projet pour les espaces universitaires, les perspectives d'évolution du projet ChillZone s'articulent autour de plusieurs axes de développement stratégiques. L'optimisation de la gestion ADE émerge comme un chantier prioritaire à explorer, car cette évolution technique, une fois mise en œuvre, permettra d'améliorer de manière significative la synchronisation des emplois du temps.

Dans le cadre du projet, l'intégration de la réalité augmentée, bien que provisoirement écartée pour respecter les délais imposés, pourrait s'inscrire dans la continuité du POC. Par ailleurs, le développement d'un projet de réalité augmentée par des étudiants de l'IUT de Meaux ouvre des perspectives prometteuses pour réaliser cette fonctionnalité.

Sur le plan infrastructurel, l'absence de matériel a été contournée mais l'installation des équipements spécifiques (frigos, box) ainsi que les serveurs pour déployer l'application sont nécessaires au fonctionnement optimal de l'application.

Ce projet a constitué une opportunité formatrice à la fois sur le plan technique et méthodologique. En effet, il a permis l'acquisition d'un ensemble de compétences pratiques, notamment des outils et des langages de programmation, mais aussi sur le plan méthodologique par la mise en application des méthodologies de gestion de projet enseignées lors de la formation. De plus, il a offert un cadre réel et stimulant pour la mise en œuvre des connaissances théoriques, en confrontant les étudiants aux exigences concrètes d'un projet collaboratif avec une Maîtrise d'Ouvrage (MOA). Cette expérience a favorisé le développement de l'esprit d'initiative, tout en renforçant notre rigueur et notre capacité à travailler en équipe. Cela nous a permis de mener à bien un projet dans un cadre à la fois académique et professionnel. Nous espérons que ce projet sera suivi et développé à partir de notre documentation et code fournis lors de la passation de projet.

Merci d'avoir lu notre rapport, nous espérons que celui-ci puisse vous servir à la fois de guide lors du développement concret de l'application "Mon IUT" mais également de support pour mieux comprendre le projet dans sa globalité ainsi que le travail fourni par notre équipe de développement.

Annexes

Ressources complémentaires (lien vers les documents annexes)

[1 - Cahier des charges fonctionnel](#)

[2 - Architecture et choix techniques](#)

[3 - Documentation de l'architecture technique](#)

[4 - Documentation base de données](#)

[5 - Maquette du projet](#)

[6 - Backlog JIRA contenant les user stories](#)

[7 - Guide d'installation du projet](#)

[8 - Documentation des tests utilisateurs](#)

[9 - Statistiques du groupe](#)

[10 - Playlist des vidéos du guide d'utilisation](#)